

A Hash Function Proposal **FEILIAN**

Submitted to the Next-generation Cryptographic Algorithms
Program, China

Designers/Submitters:

Lei Wang - Shanghai Jiao Tong University, Shanghai, China

Ling Song - Jinan University, Guangzhou, China

Yaobin Shen - Xiamen University, Xiamen, China

Kaixuan Wang - Shanghai Jiao Tong University, Shanghai, China

Zicheng Shi - Shanghai Jiao Tong University, Shanghai, China

Xin Yi - Shanghai Jiao Tong University, Shanghai, China

Email: wanglei_hb@sjtu.edu.cn

June 30, 2026

Chapter 1

Introduction

Since February, 2025, the Institute of Commercial Cryptography Standards (ICCS), China, has been launching the Next-generation Commercial Cryptographic Algorithms Program (NGCC) to promote the standardization of the next-generation commercial cryptographic algorithms, calling for proposals of next-generation cryptographic hash algorithms (NGCC-CH).

In particular, in response to the threat of quantum computing, the security requirements on hash function algorithm candidates are 512-bit collision resistance, and 1024-bit (second) preimage resistance. As a comparison, the current standardized hash functions such as SHA-2 and SHA-3 provide at most 256-bit collision resistance and 512-bit (second) preimage resistance.

FEILIAN is a candidate submitted to NGCC. Briefly, its domain extension adopts HAIFA mode [4], and its compression function adopts Addition-Rotation-XOR (ARX) design. The goal is towards achieving simplicity, transparency, security and efficiency across software and hardware platforms.

Roadmap of the Document

Chapter 2 is to provide the specification of FEILIAN.

Chapter 3 is to provide security analysis of FEILIAN.

Chapter 4 is to illustrate the design rationale and features of FEILIAN.

Chapter 5 is to report the implementation and performance of FEILIAN.

Chapter 6 is to discuss about the applications of FEILIAN.

Notations

- $+$: Addition modulo 2^{64}
- $\oplus$: Boolean exclusive or
- $\gg x$: Right rotation of x bits
- $a\|b$: Concatenation of two variables a and b
- $\|m\|$: The bit length of m
- $\leftarrow$: Variable assignment
- $\tilde{S}(\cdot)$: Tweakable S-Box
- $SC(\cdot)$: **SubColumn** operation
- $SR(\cdot)$: **ShiftRow** operation
- $AC(\cdot)$: **AddConstant** operation
- $R(\cdot)$: Round function without absorbing message block
- $R^{\oplus}(\cdot)$: Round function with absorbing message block
- $P(\cdot)$: Phase function
- $\text{FEILIAN-}f$: Compression function of **FEILIAN** hash family
- **ver**: Indicate the instance of **FEILIAN** family
- **flag**: Indicate whether a message block is the last block of message being hashed

Chapter 2

Specification

This chapter defines **FEILIAN**. It can handle messages with a bit length less than 2^{128} and its output is 1024-bit. **FEILIAN** family uses the same compression function denoted as **FEILIAN- f** , and merely differ in a parameter **ver** indicating the bit length of final hash digest.

2.1 Constants

FEILIAN uses an initial vector (IV) of sixteen 64-bit constants as below. These constants are the first decimal digits of π .

$$IV = \begin{pmatrix} IV_0, & IV_1, & IV_2, & IV_3 \\ IV_4, & IV_5, & IV_6, & IV_7 \\ IV_8, & IV_9, & IV_{10}, & IV_{11} \\ IV_{12}, & IV_{13}, & IV_{14}, & IV_{15} \end{pmatrix}$$

IV_0	$= 0x243F6A8885A308D3;$	IV_8	$= 0x9216D5D98979FB1B;$
IV_1	$= 0x13198A2E03707344;$	IV_9	$= 0xD1310BA698DFB5AC;$
IV_2	$= 0xA4093822299F31D0;$	IV_{10}	$= 0x2FFD72DBD01ADFB7;$
IV_3	$= 0x082EFA98EC4E6C89;$	IV_{11}	$= 0xB8E1AFED6A267E96;$
IV_4	$= 0x452821E638D01377;$	IV_{12}	$= 0xBA7C9045F12C7F99;$
IV_5	$= 0xBE5466CF34E90C6C;$	IV_{13}	$= 0x24A19947B3916CF7;$
IV_6	$= 0xC0AC29B7C97C50DD;$	IV_{14}	$= 0x0801F2E2858EFC16;$
IV_7	$= 0x3D84D5B5B5470917;$	IV_{15}	$= 0x636920D871574E69;$

FEILIAN uses four 64-bit constants (**Const**) as below. These constants are the first sixty-four bits of the fractional parts of the square roots of the first four prime numbers

$$\text{Const} = \left(C_0, C_1, C_2, C_3 \right)$$

$$\begin{aligned} C_0 &= 0\text{x}6\text{A}09\text{E}667\text{F}3\text{BCC}908; & C_2 &= 0\text{x}3\text{C}6\text{E}\text{F}372\text{F}\text{E}94\text{F}82\text{B}; \\ C_1 &= 0\text{x}\text{B}\text{B}67\text{A}\text{E}8584\text{C}\text{A}\text{A}73\text{B}; & C_3 &= 0\text{x}\text{A}54\text{F}\text{F}53\text{A}5\text{F}1\text{D}36\text{F}1; \end{aligned}$$

2.2 Compression Function

2.2.1 Input and Output

The input of FEILIAN compression function consists of

- a **hash chaining state** h : 1024-bit

It is divided into sixteen 64-bit variables:

$$h \longrightarrow h_0 \| h_1 \| \cdots \| h_{14} \| h_{15}.$$

It is represented as a 4×4 matrix and filled as below:

$$h = \begin{pmatrix} h_0, & h_1, & h_2, & h_3 \\ h_4, & h_5, & h_6, & h_7 \\ h_8, & h_9, & h_{10}, & h_{11} \\ h_{12}, & h_{13}, & h_{14}, & h_{15} \end{pmatrix}$$

- a **message block** m : 1024-bit

It is divided into sixteen 64-bit variables

$$m \longrightarrow m_0 \| m_1 \| \cdots \| m_{14} \| m_{15}.$$

- a **tweak** parameter tw : 256-bit

it consists of three parameters.

- a **version parameter** ver : 64-bit

indicates the instance version of FEILIAN family.

$$\begin{aligned} \text{FEILIAN-512 :} & \quad \text{ver} = 0\text{x}200 = 512; \\ \text{FEILIAN-768 :} & \quad \text{ver} = 0\text{x}300 = 768; \\ \text{FEILIAN-1024 :} & \quad \text{ver} = 0\text{x}400 = 1024; \end{aligned}$$

- a **final-call flag** `flag`: 64-bit

indicates whether the current execution of the compression function is the final execution or not.

Non-final execution :	<code>flag = 0x0000000000000000;</code>
final execution :	<code>flag = 0xFFFFFFFFFFFFFFFF;</code>

- a **counter** `t`: 128-bit

indicates the number of message bits that have been processed.

It is divided into two 64-bit variables:

$$t \longrightarrow t_0 || t_1.$$

These parameters are represented as a group of 64-bit variables as below:

$$tw = \left(\text{ver}, \text{flag}, t_0, t_1 \right)$$

The output of **FEILIAN** compression function is

- an updated hash chaining state h' : 1024-bit

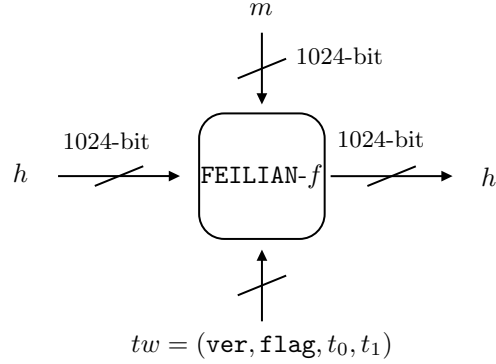


Figure 2.1: **FEILIAN** compression function

2.2.2 Basic Operations

FEILIAN-f uses several basic operations.

- **Tweakable S-Box** operations $\tilde{S}_0(\cdot, \cdot, \cdot)$ and $\tilde{S}_1(\cdot, \cdot, \cdot)$:

each tweakable S-Box operation takes two 64-bit variables (a, b) and a 64-bit tweak variable s as input, and produces updated (a', b') .

The operation $\tilde{S}_0(a, b, s)$ is as below:

$$\begin{aligned} a &\leftarrow a + b \\ b &\leftarrow b \oplus s \\ b' &\leftarrow b \ggg 8 \\ a' &\leftarrow a + \Sigma_0(b') \end{aligned}$$

where $\Sigma_0(b) = b \oplus (b \ggg 5) \oplus (b \ggg 48)$.

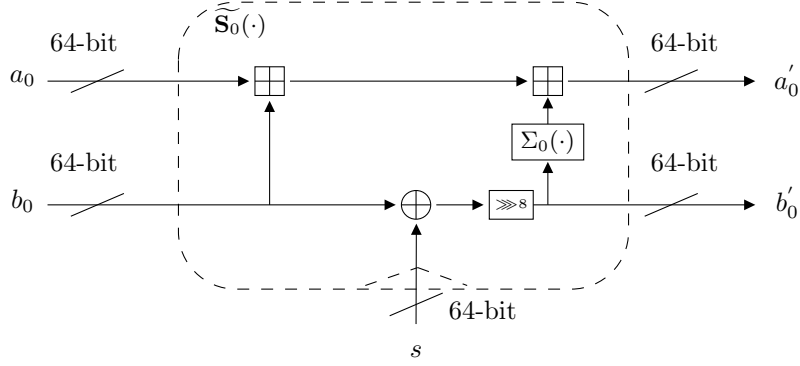


Figure 2.2: Pictorial description of $\tilde{S}_0$

The operation $\tilde{S}_1(a, b, s)$ is as below:

$$\begin{aligned} a &\leftarrow a + b \\ b &\leftarrow b \oplus s \\ b' &\leftarrow b \ggg 63 \\ a' &\leftarrow a + \Sigma_1(b') \end{aligned}$$

where $\Sigma_1(b) = b \oplus (b \ggg 11) \oplus (b \ggg 40)$

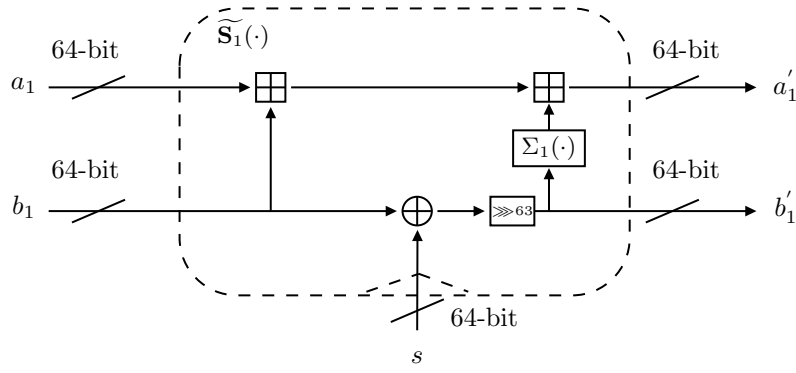


Figure 2.3: Pictorial description of $\tilde{S}_1$

- **SubColumn** operation $SC(\cdot, \cdot, \cdot, \cdot)$:

it takes four 64-bit variables (a_1, b_1, a_0, b_0) as input, and produces updated (a'_1, b'_1, a'_0, b'_0) . Essentially, it combines $\tilde{S}_0(\cdot)$ and $\tilde{S}_1(\cdot)$, referred as

$$SC(\cdot) \leftarrow \tilde{S}_0(\cdot) \otimes \tilde{S}_1(\cdot).$$

Specifically, the operation $SC(a_1, b_1, a_0, b_0)$ is as below:

$$a_1 \leftarrow a_1 + b_1$$

$$s_0 \leftarrow a_1$$

$$a_0 \leftarrow a_0 + b_0$$

$$b_0 \leftarrow b_0 \oplus s_0$$

$$b'_0 \leftarrow b_0 \ggg 8$$

$$a'_0 \leftarrow a_0 + \Sigma_0(b'_0)$$

$$s_1 \leftarrow a'_0$$

$$b_1 \leftarrow b_1 \oplus s_1$$

$$b'_1 \leftarrow b_1 \ggg 63$$

$$a'_1 \leftarrow a_1 + \Sigma_1(b'_1)$$

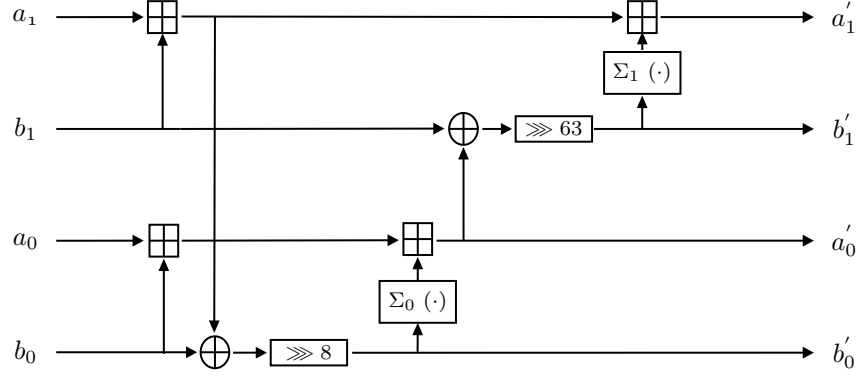


Figure 2.4: Pictorial description of SubColumn

- **ShiftRow** operation $SR(\cdot)$:

it operates on a 4×4 state matrix of 64-bit variables, and cyclically shifts the i -th row of the matrix to the left by i positions for $i \in \{0, 1, 2, 3\}$.

Specifically, **ShiftRow** is operated as below:

$$\begin{pmatrix} v_0, & v_1, & v_2, & v_3 \\ v_5, & v_6, & v_7, & v_4 \\ v_{10}, & v_{11}, & v_8, & v_9 \\ v_{15}, & v_{12}, & v_{13}, & v_{14} \end{pmatrix} \xleftarrow{SR} \begin{pmatrix} v_0, & v_1, & v_2, & v_3 \\ v_4, & v_5, & v_6, & v_7 \\ v_8, & v_9, & v_{10}, & v_{11} \\ v_{12}, & v_{13}, & v_{14}, & v_{15} \end{pmatrix}$$

- **AddConstant** operation $AC(\cdot)$:

it operates on a 4×4 state matrix of 64-bit variables, and updates it by XORing a constant state derived from **Const** and tw .

Specifically, **AddConstant** is operated as below:

$$\begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} \xleftarrow{AC} \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} \oplus \begin{pmatrix} 0 & 0 & 0 & 0 \\ \mathbf{ver}, & \mathbf{flag}, & t_0, & t_1 \\ 0 & 0 & 0 & 0 \\ C'_0 & C'_1 & C'_2 & C'_3 \end{pmatrix}$$

where (C'_0, C'_1, C'_2, C'_3) are obtained by cyclically shift $\mathbf{Const} = (C_0, C_1, C_2, C_3)$.

2.2.3 Round Function and Phase

FEILIAN- f uses two types of round functions:

- $R^{\hat{\oplus}}(\cdot, \cdot)$: absorb message and update chaining state;
- $R(\cdot)$: update chaining state.

Round Function $R^{\hat{\oplus}}(\cdot, \cdot)$

It operates on a 4×4 state matrix of 64-bit variables and a 1024-bit message block. It absorbs message half by half, and the absorption is denoted as $\hat{\oplus}$. We take the first half message as an example, which is operated as below:

$$\begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} \xleftarrow{\hat{\oplus}} \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} \oplus \begin{pmatrix} m_0 & m_1 & m_2 & m_3 \\ 0 & 0 & 0 & 0 \\ m_4 & m_5 & m_6 & m_7 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Specifically, the operation $R^{\hat{\oplus}}(\cdot, \cdot)$ is as below:

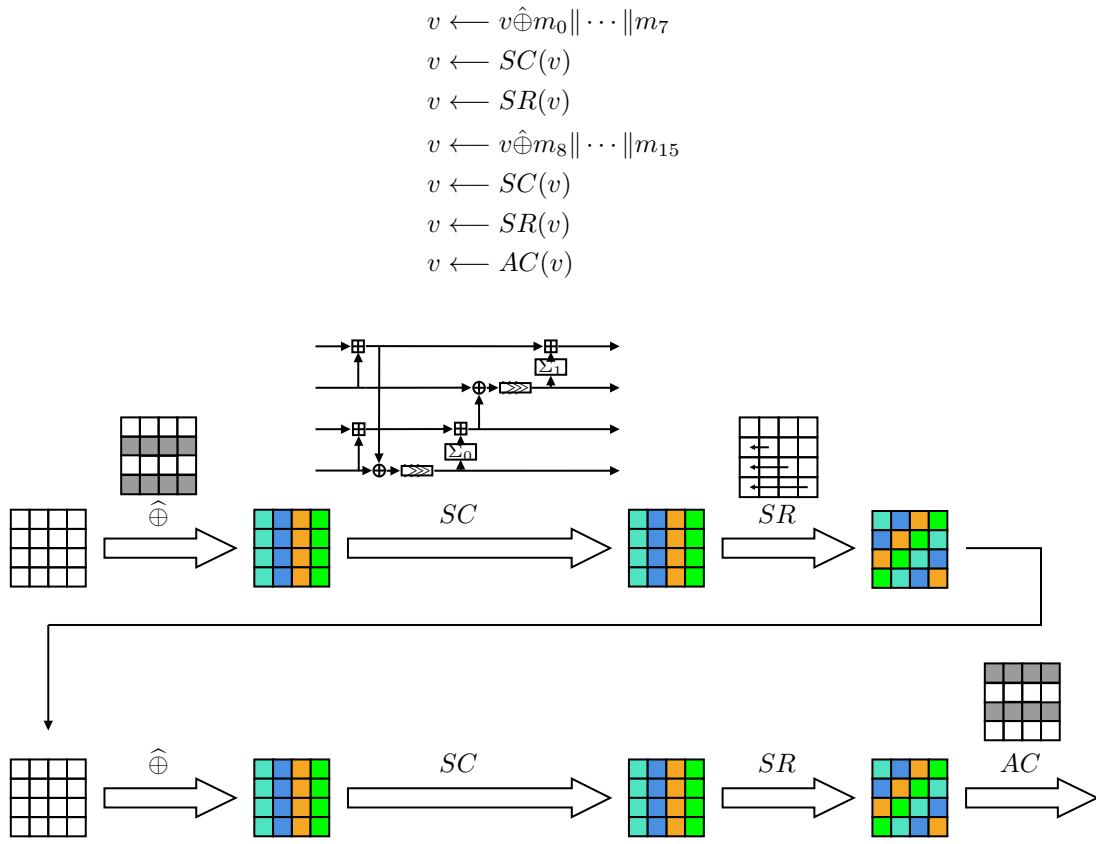


Figure 2.5: Pictorial description of $R^{\hat{\oplus}}$

Round Function $R(\cdot)$

It operates on a 4×4 state matrix of 64-bit variables. Specifically, the operation $R(\cdot)$ is as below:

$$\begin{aligned} v &\leftarrow SC(v) \\ v &\leftarrow SR(v) \\ v &\leftarrow SC(v) \\ v &\leftarrow SR(v) \\ v &\leftarrow AC(v) \end{aligned}$$

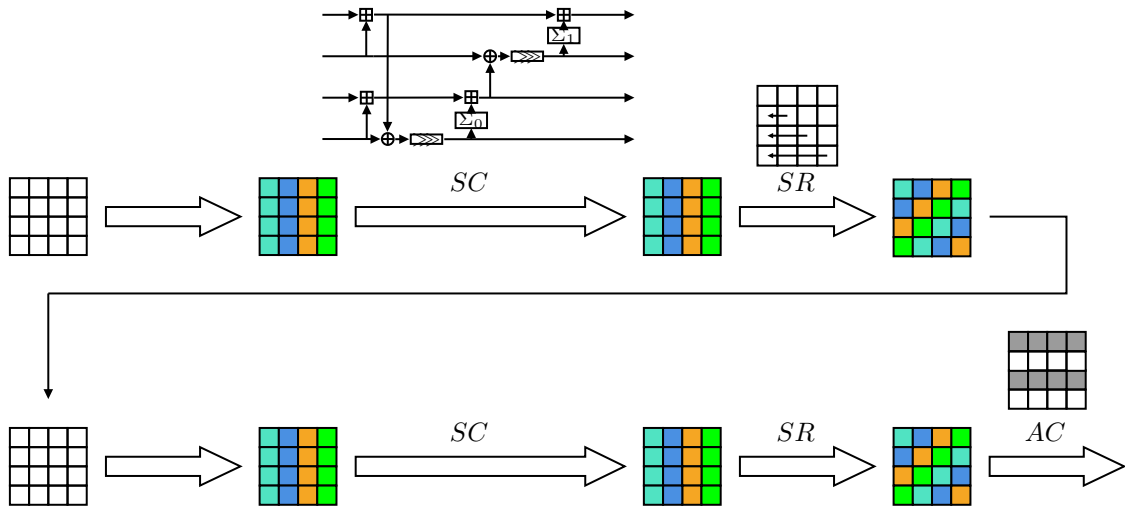


Figure 2.6: Pictorial description of R

Phase $P(\cdot, \cdot)$

It operates on a 4×4 state matrix of 64-bit variables and a 1024-bit message block. It consists of four round functions to absorb message and update the state.

Specifically, the operation $P(v, m)$ is as below:

$$\begin{aligned} v &\leftarrow R^{\oplus}(v, m) \\ v &\leftarrow R(v) \\ v &\leftarrow R(v) \\ v &\leftarrow R(v) \end{aligned}$$

Constant for $AC(\cdot)$: at the i -th phase, (C'_0, C'_1, C'_2, C'_3) is set by cyclically shift **Const**:

$$(C'_0, C'_1, C'_2, C'_3) = (C_{i\%4}, C_{(i+1)\%4}, C_{(i+2)\%4}, C_{(i+3)\%4}).$$

2.2.4 Compression Function

The compression function consists of five phases (in turn 20 rounds). Specifically, **FEILIAN- f** is operated as follows.

$$\begin{aligned} v_0 &\leftarrow h \\ v_1 &\leftarrow P_0(v_0, m) \\ v_2 &\leftarrow P_1(v_1, m) \\ v_3 &\leftarrow P_2(v_2, m) \\ v_4 &\leftarrow P_3(v_3, m) \\ v_5 &\leftarrow P_4(v_4, m) \\ h' &\leftarrow v_0 \oplus v_5 \end{aligned}$$

2.3 Hash Function

FEILIAN hashes messages of bit length less than 2^{128} . Let M be an input message.

Splitting Message

M is split into 1024-bit blocks:

$$M \longrightarrow M_0 \| M_1 \| \cdots \| M_\ell.$$

If the last block M_ℓ is not full, namely its bit length is less than 1024, pad bits '0's to M_ℓ until it becomes 1024 bits.

$$M'_\ell \leftarrow M_\ell \| 0 \cdots 0.$$

If the last block M_ℓ is full block, namely its bit length is 1024.

$$M'_\ell \leftarrow M_\ell.$$

Setting Parameters

The parameter **ver** is set as below:

$$\mathbf{ver} = 0\mathbf{x}200/0\mathbf{x}300/0\mathbf{x}400; \quad \text{for FEILIAN-512/768/1024}$$

Iterating Compression Function

The above message blocks is processed via a sequential iteration as below:

```

 $h_0 \leftarrow IV$ 
 $t \leftarrow 0$ 
 $\mathbf{flag} \leftarrow 0\mathbf{x}0000000000000000$ 
for  $i = 0, \dots, \ell - 1$ 
     $t \leftarrow t + 1024$ 
     $tw \leftarrow (\mathbf{ver}, \mathbf{flag}, t)$ 
     $h_{i+1} \leftarrow \text{FEILIAN-}f(h_i, M_i, tw)$ 
end for
 $t \leftarrow t + \|M_\ell\|$ 
 $\mathbf{flag} \leftarrow 0\mathbf{x}\text{FFFFFFFFFFFFFFFF}$ 
 $tw \leftarrow (\mathbf{ver}, \mathbf{flag}, t)$ 
 $h_{\ell+1} \leftarrow \text{FEILIAN-}f(h_\ell, M'_\ell, tw)$ 
return  $h_{\ell+1}$ 

```

Producing Hash Digest

$h_{\ell+1}$ is represented as a 4×4 state matrix:

$$h_{\ell+1} = \begin{pmatrix} h_0, & h_1, & h_2, & h_3 \\ h_4, & h_5, & h_6, & h_7 \\ h_8, & h_9, & h_{10}, & h_{11} \\ h_{12}, & h_{13}, & h_{14}, & h_{15} \end{pmatrix}$$

The final digest h of message M is computed as below, with respect to concrete instances of **FEILIAN** family.

$$\begin{aligned} \text{FEILIAN-512 : } h &= h_0 \| h_1 \| \dots \| h_7; \\ \text{FEILIAN-768 : } h &= h_0 \| h_1 \| \dots \| h_{11}; \\ \text{FEILIAN-1024 : } h &= h_0 \| h_1 \| \dots \| h_{15}; \end{aligned}$$

Chapter 3

Design Rationale and Features

3.1 Design Rationale

Narrow-State, Single Compression Function, and Simplicity

- **Choosing HAIFA mode:** as the core requirement of NGCC call-for-proposal, FEILIAN-1024 aims to provide as high as 512/1024-bit security for collision/preimage resistance. It implies that the chaining state should be at least 1024 bits. Clearly, larger-size state consumes computation and memory resources. Thus, FEILIAN-1024 uses a narrow-pipe chaining state, that is exactly 1024-bit. Under such circumstance, FEILIAN adopts HAIFA mode which is narrow-pipe and can resist generic second-preimage attacks. HAIFA mode is proven indifferentiable from random oracle, hence supporting various hash-based applications.
- **Single function:** all FEILIAN family make use of the same compression function FEILIAN- f , merely differing in a parameter variable `ver` to indicate the concrete instances. An implementation of the core compression function on a platform enables supporting all FEILIAN hash family. This is engineer and platform friendly, reducing the workload of (optimizing) implementation.
- **Simplicity and symmetry:** FEILIAN can be regarded as being built *solely* on a tweakable S-box structure of basic and common operations Addition, Rotation and bit-wise XOR. The compression function makes composition of such S-boxes, *i.e.*, two pair-wised S-boxes composing SubColumn and four parallel SubColumn composing SubMatrix. FEILIAN is expected to be fully described and hence completely rememberable, via few lines of natural language description. Towards this extreme simplicity goal, FEILIAN use no message word re-ordering or linear expansion. Both initial vector and `const` can be described via natural language, namely one does not need to memorize it. Besides the graphical structure, it seems that dedicated parameters of FEILIAN one has to memorize is merely the six rotation numbers in $\tilde{S}_0$ and $\tilde{S}_1$.

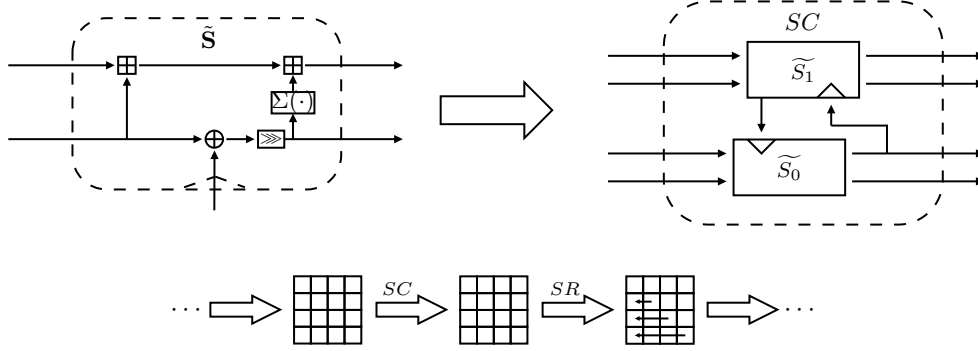


Figure 3.1: Composing FEILIAN via simple ARX S-box

Tradeoff on Security Transparency and Margin

Due to high-efficiency requirements, a cryptographic hash function usually does not have a security reduction to a hard mathematical problem, and hence the security is heuristic. On the other hand, the lack of an explicit security basis incurs hash function designs with overly conservative security margin [3].

FEILIAN iterates internal permutations that are composed of several consecutive rounds, referred to *phases* in the specification. Hence, the message words are injected to update state every several rounds rather than every round.

FEILIAN lets such internal permutations act as security “basis” for the overall structure, to certain extent. Estimating the security properties of these internal permutations can well imply the security strength of FEILIAN- f and in turn the whole hash function. In particular, without taking message injection into consideration (eliminating the impact of message modification/adaptively selection fully controlled by attackers), the state transition enables more transparent security properties due to the symmetry. We refer to section 4 for detailed discussion. In turn, a reasonable security margin is chosen for security and efficiency tradeoff.

Parallelism for Performance and Flexibility across Platforms

FEILIAN is strongly parallelizable, specifically, four SubColumn can be operated in parallel. It enables efficient software implementation on mainstream 64-bit platforms in SIMD (Single instruction, multiple-data) manner. Moreover, such parallel structure enables FEILIAN very flexible for hardware platforms, since it can be folded both vertically and horizontally in one round. We refer to section 5 for detailed discussion.

3.2 Choosing Dedicated Parameters

- **Little-endian:** a majority of 64-bit platforms are little -endian, and thus FEILIAN is little-endian;
- **Rotation numbers:** we have performed an automated search among plausible combinations

of rotation numbers for **Subcolumn** operation, with respect to both the security and efficiency concerns. Note that the space is too large to exhaustively search. We have narrowed down search space from performance concern, e.g., rotating by a multiple of 8 bits consumes just one instruction.

- **Phase/Round number:** we have made efforts to investigate the security properties of round function and consecutive round functions (*i.e.*, phase function). We refer to section 4 for details. In the end, We choose to group four rounds as a phase and then iterate five phases for **FEILIAN- f** .

3.3 Features

We summarize and highlight the features of **FEILIAN** hash family.

Design:

- simplicity for the algorithm
- a single compression function for all **FEILIAN** hash family

Security:

- resistance to length-extension and second preimage attacks
- tradeoff on transparency and margin

Performance:

- fast in both software and hardware
- flexibility for hardware implementation

Chapter 4

Security

4.1 Security Claims

We provide security claims of FEILIAN hash family in the classical setting as in Table 4.1, Moreover, we claim that there is no dedicated quantum attack on FEILIAN hash family with respect to the fundamental security notions, that is collision resistance, second preimage resistance and preimage resistance, with less complexity than that of generic quantum attacks applicable on any hash function.

Table 4.1: The security claims of FEILIAN hash family in classical setting.

	Collision	Second-preimage	Preimage
FEILIAN-1024	512 bits	1024 bits	1024 bits
FEILIAN-768	384 bits	768 bits	768 bits
FEILIAN-512	256 bits	512 bits	512 bits

4.2 Randomness Statistical Test

We have performed NIST (National Institute of Standards and Technology) SP 800-22 test [1] to FEILIAN. The results have been shown in Table 4.2, which well demonstrates the good randomness of FEILIAN outputs.

Table 4.2: 50M-bit NIST STS Randomness Test Results — All Passed

No.	Test Item	P-value	Result
01	Frequency	0.349020	✓
02	Block Frequency	0.506258	✓
03	Cumulative Sums	0.585879 / 0.175345	✓
04	Runs	0.032921	✓
05	Longest Run of Ones	0.596658	✓
06	Rank	0.035764	✓
07	FFT	0.216170	✓
08	NonOverlapping Template	All 148 items passed	✓
09	Overlapping Template	0.187710	✓
10	Universal	0.455079	✓
11	Approximate Entropy	0.315981	✓
12	Random Excursions	8 items, minimum 0.393089	✓
13	Random Excursions Variant	18 items, minimum 0.285659	✓
14	Serial	0.691705 / 0.899559	✓
15	Linear Complexity	0.652984	✓

Test Description: Randomness test results of 50M-bit output from FEILIAN hash function. A test item is passed if P-value ≥ 0.01

0/1 Distribution: 25,003,311 / 24,996,689 $\approx$ 50.007% / 49.993%

4.3 Security of Hash Mode

HAIFA [4] is a mode of operation for constructing hash functions. It is similar to Merkle-Damgård construction [14, 7] but with a salt and a counter. FEILIAN employs a customized HAIFA variant that relies solely on a counter without a salt. Besides, a **flag** word is used to indicate whether the message block being processed is the final one. FEILIAN pads the last block with bits ‘0’s only when necessary. Specifically, no padding is added if the input data length is an integer multiple of the block size. The security requirement originally guaranteed by traditional padding is thus undertaken by this **flag**. The proposed HAIFA variant retains the inherent security properties of the original HAIFA, including resistance to generic attacks, e.g., long-message second-preimage attacks and length extension attacks.

4.3.1 Indifferentiability

Let $C : \{0, 1\}^n \times \{0, 1\}^b \times \{0, 1\}^w \times \{0, 1\}^v \rightarrow \{0, 1\}^n$ be an ideal compression function modeled as a random oracle. Let $H^C : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be the HAIFA hash function utilizing C . The internal state is updated as $h_i = C(h_{i-1}, M_i, t_i, s)$, where t_i is the number of bits hashed so far, and s is the salt. The padding rule ensures that the last block encodes the message length. The HAIFA construction H^C is indistinguishable from a random oracle $RO : \{0, 1\}^* \rightarrow \{0, 1\}^n$. For any distinguisher $\mathcal{D}$ making at most q queries, the advantage is bounded by $\mathcal{O}(q^2/2^n)$. To prove indistinguishability, we construct a simulator S such that $\mathcal{D}$ cannot distinguish between (H^C, C) and (RO, S^{RO}) . S processes queries to the compression function C and maintains a directed tree of states T , rooted at the initial value IV .

Algorithm for Simulator S on input (h, m, t, s) :

Table Lookup: If the query (h, m, t, s) is in the local table, return the previously stored output y .

Check Path: Check if $h \in T$ (i.e., h is reachable from IV). If $h \notin T$, the query is isolated. Sample $y \leftarrow \$\{0, 1\}^n$. If $h \in T$, extract the unique message prefix M_{pre} corresponding to the path from IV to h .

Padding and Counter Validation: Verify if m contains a valid HAIFA padding and if the counter t matches the bit length of $M = M_{pre} \parallel m$.

Response Generation: If Step 3 is valid (it is a legal final block): Query the random oracle for the full message, $y \leftarrow RO(M)$. Otherwise (it is an intermediate block): Sample $y \leftarrow \$\{0, 1\}^n$.

State Update: Store $(h, m, t, s) \mapsto y$. Add y to T as a child of h . Return y .

The views of $\mathcal{D}$ in (H^C, C) and (RO, S^{RO}) are identical unless one of the following BAD events occurs in the simulated world.

Event B_1 (Internal Collision): S generates a random value y that is equal to a previously generated value, or equal to the IV . Since S generates at most q random values of length n , by the union bound, $\Pr[B_1] \leq \frac{q(q-1)}{2 \cdot 2^n} + \frac{q}{2^n}$.

Event B_2 (Out of Order): $\mathcal{D}$ queries S with a state $h \in T$ that was not obtained as an output of a previous query to S . The probability of guessing a random n -bit output is bounded by $\Pr[B_2] \leq \frac{q}{2^n}$. The core of the proof relies on the fact that HAIFA is prefix-free. The counter t_i and the message length encoded in the padding guarantee that no valid final block can be processed as an intermediate block of another message. Consequently, S will never incorrectly assign a random

value to a final block, nor will it assign an *RO* query result to an intermediate block. Assuming $\neg(B_1 \cup B_2)$, the directed graph T is a set of trees (no cycles), and the simulation is perfectly consistent with *RO*. The distinguisher’s advantage is bounded by the probability of the bad events. $\text{Adv}_{\mathcal{D}} \leq \Pr[B_1 \cup B_2] \leq \mathcal{O}(q^2/2^n)$. This completes the proof.

4.3.2 Generic Attacks

Length extension. The length extension is finding two distinct messages such that the computation of one message is a prefix of the computation of the other message, implying that the hash digest of the former message is an internal hash chaining state of the latter message. Such a pair of messages leads to distinguishing and forgery attacks.

The mode of **FEILIAN** uses a finalization indicator **flag**. The **flag** word is set to `0xFFFFFFFFFFFFFFFF` if the message block being processed is the last, and to `0x0000000000000000` otherwise. Hence, the execution on the last block of a message differs in **flag** from any intermediate compression function of other messages. If **FEILIAN** is collision resistant, then the length extension attack does not apply to **FEILIAN**. Thus, the structure is immune to the length extension attack.

Long-message second preimage attack. A second-preimage attack aims to construct a distinct message $M' \neq M$ that produces the same hash digest as a given target message M . Kelsey and Schneier put forward a second-preimage attack on Merkle-Damgård hash functions (with MD strengthening) with a complexity of $k \times 2^{n/2+1} + 2^{n-k+1}$ for a 2^k -block message [12]. The core building element of this attack is a structure of expandable messages: a set of messages with different lengths that all compress to the identical intermediate chaining value, which are obtained by exploiting Joux’s multi-collisions [10]. When the expandable messages are constructed, a message of any block length ranging from k to $k + 2^k - 1$ can be retrieved leading to the same intermediate chaining value. There is a fundamental requirement for constructing expandable messages via multi-collisions that the compression function should be the same with any number of bits processed before. The **FEILIAN** hash function encodes the number of processed bits that tweaks the compression function. This number of bits encoding feature invalidates the construction of expandable messages, which is the fundamental prerequisite for the second-preimage attack for a long message proposed by Kelsey and Schneier. Therefore, the structure of **FEILIAN** is immune to the Kelsey and Schneier’s second-preimage attack.

Another second-preimage attack [9] on Merkle-Damgård hash mode is to build a tree-like multi-collision structure, that is a set of chaining variables hashing to the same root state via different messages with the equal block length. There is fundamental requirement that the root must be able to be linked to various block positions of the original message. Again, the number of processed bits encoding invalidates such a linking. Thus, such second-preimage attacks cannot be applicable.

Multi-collision attack. Joux’s multi-collision attack [10] is to find collisions consecutively on message blocks in a sequential manner, which enables a multi-collision with the complexity of finding one pair of collision. For example, with two pairs of blocks, M_0 and M'_0 colliding on first block and M_1 and M'_1 colliding on second block, it brings four-collision $M_0 \parallel M_1$, $M_0 \parallel M'_1$, $M'_0 \parallel M_1$ and $M'_0 \parallel M'_1$. As **FEILIAN** has a narrow-pipe state, Joux’s multi-collision is applicable. Given this attack is based on a Prerequisite that one can find collisions on **FEILIAN**, it should not be considered as a real threaten as long as **FEILIAN** is collision resistance.

Herding attack. Herding attack [11] is to pre-compute a tree-like multi-collision and announce the root hash. Then the attacker can find a chosen-prefix message linking IV to the root hash with the aid of a tree-like multi-collision. As FEILIAN has a narrow-pipe state, herding attack is applicable. Again, such an attack is also based on a Prerequisite that one can find collisions on FEILIAN. Hence, it should not be considered as a real threat as long as FEILIAN is collision resistant.

4.4 Security of Compression Function Structure

Davies-Meyer mode. FEILIAN compression function adopts Davies-Meyer mode. It has been proven collision resistance and pre-image resistance as long as the underlying block cipher is an ideal cipher [6]. On the other hand, there is a fixed-point property on Davies-Meyer mode, namely one can find (h, m) such that $\text{FEILIAN-}f(h, M) = h$. To our best knowledge, fixed-point properties does not have real impact on hash function security (except accelerating multi-collision slightly), particularly on HAIFA mode.

Iterative Even-Mansour structure. The underlying block cipher of FEILIAN compression function can be regarded as a single-key Even-Mansour structure, as depicted in Figure 4.1. Various concrete attacks on iterative Even-Mansour have been published [15]. Among these attacks, it seems that chosen-key differential distinguisher reaches the largest attacked steps. However, we note such an attack starts with an inbound meet-in-the-middle matching, which leverages all freedom of both the message and the internal state. As a result, it cannot be applied to hash function, since linking a fixed initial value to the determined chaining value is at least birthday bound complexity. Moreover, it is not applicable to FEILIAN in a keyed application since the chaining states are secret.

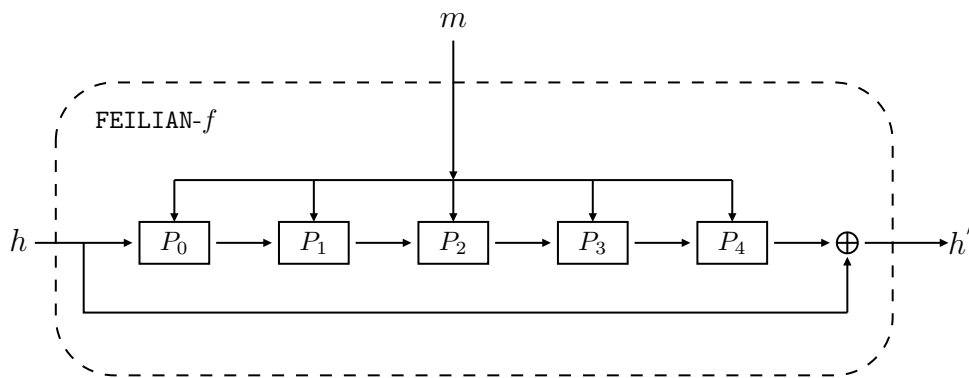


Figure 4.1: FEILIAN compression function structure

4.5 Security Properties of Components

4.5.1 Tweakable S-Box

The tweakable S-Box $\tilde{S}$ is a simple Addition-Rotation-Xor S-box, and the basic operations of $\tilde{S}$ are sequential for enhancing difference propagation. The difference will be propagated to the other

pair-wised S-box and then to other columns via **ShiftRow** operations. We refer to Section 4.5.3 for difference propagation evaluations.

We remark that there is one specific differential on $\tilde{S}_0(\cdot)$ that will not be propagated to other variables as below:

$$(\Delta a_0 = 0, \Delta b_0 \neq 0, \Delta s_0 = 0) \longrightarrow (\Delta a'_0 = 0, \Delta b_0 \neq 0).$$

Due to the prime rotation number in Σ_0 , Δb_0 will be `0xFFFFFFFFFFFFFFFF`. The differential probability is quite low for such a heavy difference. Thus, it cannot be applied to compression function with respect to its round number.

4.5.2 SubColumn

The **SubColumn** operation $(a'_1, b'_1, a'_0, b'_0) = SC(a_1, b_1, a_0, b_0)$ combines two tweakable ARX S-boxes. It is bijective and its inverse is defined as follows:

$$\begin{aligned} s_0 &\longleftarrow a'_1 - \Sigma_1(b'_1), \\ b_1 &\longleftarrow a'_0 \oplus (b'_1 \lll 63), \\ s_1 &\longleftarrow a'_0 - \Sigma_0(b'_0), \\ b_0 &\longleftarrow s_0 \oplus (b'_0 \lll 8), \\ a_1 &\longleftarrow s_0 - b_1, \\ a_0 &\longleftarrow s_1 - b_0. \end{aligned}$$

There are differentials of probability of 1 for it. For example,

$$\begin{array}{cccc} 0000000000000000 & 8000000000000000 & 0000000000000000 & 8000000000000000 \\ 8000000000000000 & 0000000000000000 & 8000000000000000 & 0000000000000000 \\ 0000000000000000 & 8000000000000000 & 0000000000000000 & 8000000000000000 \\ 8000000000000000 & 0000000000000000 & 8000000000000000 & 0000000000000000 \end{array} \longrightarrow \begin{array}{cc} & , \\ & \longrightarrow \end{array}.$$

In addition, no preservation of differences can be obtained. In other words, the zero difference is the only fixed point.

4.5.3 Consecutive Round Functions

The diffusion property describes the round function's capacity to rapidly distribute a tiny input difference over all components of the internal state. As there are probability-1 differentials for the **SubColumn** operation, the same happens for $SR \circ SC$, as below, which is defined as a half round.

$$\begin{array}{cccc} 0000000000000000 & 0000000000000000 & 0000000000000000 & 0000000000000000 \\ 8000000000000000 & 0000000000000000 & 0000000000000000 & 0000000000000000 \\ 0000000000000000 & 0000000000000000 & 0000000000000000 & 0000000000000000 \\ 8000000000000000 & 0000000000000000 & 0000000000000000 & 0000000000000000 \end{array}$$

$\downarrow SC$

```

8000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
8000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000

```

$\downarrow SR$

```

8000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 8000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000

```

The highest-probability differential trail identified over one round has a probability of 2^{-3} . As the number of rounds grows, the diffusion behavior strengthens markedly. After three rounds, a good avalanche effect is achieved. Table 4.3 lists the average number of flipped bits after three rounds for each output word, under the condition of a random single-bit input difference in the first column of the state.

Table 4.3: Average number of flipped bits in each output word after three rounds

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	31.9	32.0	31.8	32.1	32.1	32.2	32.1	31.9	32.1	32.2	32.1	31.8	32.3	31.9	32.0	32.3
1	32.0	32.1	32.2	31.8	31.9	32.2	32.0	31.9	31.7	32.0	31.8	32.2	31.9	32.0	32.0	31.7
2	31.7	31.7	31.8	32.0	32.1	32.1	32.0	31.9	31.7	32.0	32.1	32.1	31.9	32.0	32.0	32.1
3	31.9	31.9	32.1	32.2	32.0	32.1	31.8	32.2	31.9	32.2	31.7	32.4	32.3	31.7	32.0	32.2

4.6 Dedicated Attacks

4.6.1 Rotational Attack

Rotational cryptanalysis [13] is a dedicated technique for analyzing ARX constructions. It considers input pairs related by a fixed cyclic rotation, such as x and $x \lll r$. XOR and rotation preserve this relation exactly, whereas modular addition preserves it only probabilistically because of carry propagation. If the resulting probability is significantly higher than that of a random function, the relation can be used to construct a distinguisher. The injection of round constants is an important countermeasure, since non-rotation-invariant constants disrupt the rotational symmetry required by the attack.

In **FEILIAN**, the round function includes the addition of round-dependent constants, which directly prevent rotational relations from being preserved across rounds. Moreover, the number of rounds is sufficiently large to ensure that the probability of any residual rotational relation decreases rapidly

through repeated modular additions and constant injections. Consequently, the overall propagation probability is expected to be too low to support a practical distinguisher or attack. Therefore, rotational cryptanalysis is not expected to pose a practical threat to either the compression function or the complete hash function FEILIAN.

4.6.2 Slide Attack

The slide attack [5] is a cryptanalytic technique that targets constructions composed of many repetitions of an identical or highly similar transformation. Unlike differential and linear cryptanalysis, its effectiveness does not necessarily decrease as the number of rounds increases; if all rounds are structurally identical, the attacker attempts to “slide” two computations against each other so that they are shifted by one round. Thus, instead of analyzing the full-round construction directly, the attacker searches for pairs satisfying the sliding relation and reduces the analysis to a single round or repeated transformation. Once identified, such pairs may expose relations between internal states, distinguish the construction from an ideal primitive, or, in the case of block ciphers, facilitate key recovery. Slide attacks are therefore particularly relevant to constructions with identical round functions and without sufficiently strong round-dependent constants or transformations.

This is the case for hash functions whose internal permutations repeatedly apply the same transformation. In FEILIAN, constants are used in the round functions. In phases $P_i, 0 \leq i \leq 4$, $(C'_0, C'_1, C'_2, C'_3) = (C_{i\%4}, C_{(i+1)\%4}, C_{(i+2)\%4}, C_{(i+3)\%4})$, and hence the phase structure repeats after four phases, i.e., $P_0 = P_4$. Although the same constant-addition pattern is used within each phase, aligning two executions at P_0 and P_4 requires a slide over four complete phases rather than a single phase. In each phase, the round functions apply the same constant addition. To align P_0 and P_4 , one needs to “slide” four phases. It is believed that four phases are sufficient to prevent the slide attack. Consequently, a valid slid relation must propagate through four successive phases, during which nonlinear operations and state diffusion substantially reduce its probability. We therefore expect that four phases provide a sufficient security margin against practical slide attacks.

Rotational slide attack. Another possible attack is to apply rotational slide attacks. Specifically, one sets $(\text{ver}, \text{flag}) = (t_0, t_1)$. Under such a parameter setting, the goal is to find a pair of chaining states with the same message such that the state v_3 of one input is a two-column rotation of the state v_0 of the other message. Consequently, a rotational slide propagates two consecutive phases at the beginning and another two consecutive phases at the end.

4.6.3 Meet-in-the-Middle Attack

The meet-in-the-middle attack [2] is dividing message blocks into two independent chunks and searching via a meet-in-the-middle manner on the compression function to get pseudo-preimages, which can be transformed to be a preimage on the whole hash function via another meet-in-the-middle matching on the Merkle-Damgård mode.

In FEILIAN, there are five phases. Dividing messages into two independent chunks enable the attacker bypass two phases, leaving three phases in the middle, which is unlikely matchable due to the fast unknown bits propagation.

4.6.4 Boomerang Attack

The boomerang attack [16] is a differential cryptanalytic technique that combines two short differential characteristics to analyze a construction for which finding a high-probability characteristic over all rounds is difficult. Let an n -bit permutation be decomposed as $E = E_1 \circ E_0$. Suppose that the upper part E_0 has a differential $\alpha \rightarrow \beta$ with probability p , while the lower part E_1 has a differential $\gamma \rightarrow \delta$ with probability q . One can combine them to form a characteristic $\alpha \rightarrow \delta$ for quartets with probability p^2q^2 . By contrast, the corresponding relation holds for a random n -bit permutation with probability approximately 2^{-2n} . Hence, the attack may distinguish the target from a random permutation when $p^2q^2 > 2^{-2n}$.

For hash functions, boomerang techniques are generally applied to the internal permutation or compression function, where sufficiently many inputs and intermediate differences can be controlled or analyzed. Such results primarily assess the security margin of the underlying components and do not necessarily translate into collision, preimage, or other practical attacks on the complete hash function.

In FEILIAN, high-probability differential characteristics may exist over a small number of rounds, whereas their probabilities decrease substantially as the number of rounds increases. The boomerang attack on FEILIAN requires two or three consecutive active phases in one of characteristics, which provides a substantial security margin against boomerang attacks.

Chapter 5

Implementation and Performance

5.1 Performance Analysis

5.1.1 Basic Operations

A single `SubColumn` consists of four 64-bit additions, eight XOR operations and eight rotations. One full round executes two `Submatrix` (four parallel `SubColumn` transforms each) plus row shifts and constant XORs; one message-bearing round adds sixteen extra message-word XORs while non-message rounds omit these extra XORs. The compression function runs 20 rounds grouped into five four-round cycles (one message round plus three non-message rounds). After all rounds, a final state feedforward adds sixteen XORs. Summing all atomic arithmetic and rotation operations, one complete FEILIAN compression function `FEILIAN-f` contains 640 additions, 1216 XORs and 960 rotations. Padding, block loading and state copy routines only involve memory moves with negligible arithmetic overhead.

5.1.2 Memory

`FEILIAN-f` needs to store in ROM 128 bytes for the initialization vector (sixteen 64-bit words arranged as a 4×4 matrix) and 128 bytes for the four groups of phase constants (each holding four 64-bit words), thus 256 bytes in total. Consequently, no ROM storage is required for permutation tables. In RAM, the storage m (message block, sixteen 64-bit words), h (chaining value, 4×4 64-bit words), s (state, 4×4 64-bit words), t (128-bit message-bit counter occupying two 64-bit words), and the backup h (initial state copy for the final feed-forward XOR, 4×4 64-bit words) requires 528 ($= 128 + 128 + 128 + 16 + 128$) bytes. In practice, the state matrix s and the output chaining value h are the same structure in the compression function, and the backup h can be eliminated if the caller guarantees preservation of the input chaining value across the compression call, reducing the minimum RAM requirement to 400 bytes. A hardware implementation that reads the message block from an external stream rather than buffering it entirely could further reduce on-chip RAM by 128 bytes.

Memory and Speed tradeoff

A typical tradeoff is to store some additional data in memory in order to reduce the number of computation steps and in turn speed up hash function implementation. FEILIAN’s round constants admit a modest memory/speed tradeoff: the four groups of phase constants are cyclic rotations of one another. An implementation may therefore store only `const` (32 bytes) and compute the appropriate constant for each phase via an index offset, saving 96 bytes of ROM at the cost of four modular-index operations per round. That said, the absolute saving is negligible on all but the most constrained platforms.

5.1.3 Parallelism

FEILIAN- f offers parallelism at several granularities. The primary source of intra-round parallelism is the column-wise independence of **SubMatrix**: each of the four columns of the 4×4 state matrix is processed by an independent **SubColumn** call, with no cross-column data dependencies until the subsequent **ShiftRow** operation. A single **SubMatrix** call thus exposes 4-way parallelism — four **SubColumn** instances can execute concurrently, each operating on a distinct column of four 64-bit words. Within each **SubColumn**, the instruction-level parallelism is modest: the first two additions ($a += b$ and $c += d$) are independent and can issue simultaneously, and the two rotations inside each Sigma function ($\text{rotr64}(x, 5)$ and $\text{rotr64}(x, 48)$ in Σ_0 ; $\text{rotr64}(x, 11)$ and $\text{rotr64}(x, 40)$ in Σ_1) are likewise independent of each other. Beyond these pairs, the column update follows a tight dependency chain ($a \leftrightarrow b \rightarrow c \rightarrow d \rightarrow a$) that serializes the remaining operations.

In SIMD architectures, the column structure maps naturally to vector widths: one column comprises four 64-bit words (256 bits), fitting exactly into a single 256-bit SIMD register (e.g., AVX2). With four such registers, all four columns of a **SubMatrix** can be processed simultaneously using vectorized addition, XOR, and rotation instructions. The **ShiftRow** permutation, however, requires cross-register element rearrangement (reshuffling words between the four SIMD registers), which on some architectures may incur a non-trivial latency. Wider SIMD units (e.g., 512-bit AVX-512) do not natively double the column throughput — since a column is intrinsically 256 bits wide — but can be exploited by hashing two independent messages in lockstep within the same instruction stream.

5.2 Software Implementation Techniques

The SIMD-accelerated FEILIAN- f implementation employs a number of engineering optimization techniques, spanning from algorithmic-level round structure simplification to micro-architectural instruction scheduling. We summarize the key techniques below.

5.2.1 AVX2 SIMD vectorization of the 4×4 State Matrix

The compression function maintains a 4×4 matrix of 64-bit words as its internal state. Rather than operating on 16 individual 64-bit scalars, each row of the matrix is packed into a single 256-bit AVX2 register (`_m256i`), yielding four SIMD vectors `r0–r3`. All arithmetic—additions, XORs, and rotations—operate on four 64-bit lanes in parallel with single AVX2 instructions. This reduces the per-round instruction count by a factor of four compared to a scalar implementation and eliminates the latency of serial lane processing.

5.2.2 Hardware Shuffle-based Rotations via VPSHUFB

A central performance bottleneck in ARX (Add–Rotate–XOR) hashes is the rotation operation, which on AVX2 normally requires a shift-right, a shift-left, and an OR to compose the rotated result. For byte-granularity rotations (multiples of 8 bits), the implementation replaces the three-instruction `shift+shift+or` sequence with a single `VPSHUFB` (`_mm_shuffle_epi8`) instruction. Five static 32-byte permutation masks are precomputed with 32-byte alignment:

Rotation	Mask	Cost (instructions)
ROT8	<code>rot8_mask</code>	1 (<code>VPSHUFB</code>)
ROT40	<code>rot40_mask</code>	1 (<code>VPSHUFB</code>)
ROT48	<code>rot48_mask</code>	1 (<code>VPSHUFB</code>)

Each mask encodes the byte-level permutation that implements an n -byte left-rotation within each 64-bit lane, while keeping the high and low lanes symmetric. The mask is loaded once via a `_m256i` cast of the aligned static array, and the rotation becomes a single-cycle port-5 operation, significantly reducing both latency and pressure on the vector ALU ports.

5.2.3 Clever Encoding of ROT63 via Add-self Trick

For the 63-bit left rotation, the conventional approach would be:

$$\text{ROT63}(x) = (x \gg 63) \mid (x \ll 1)$$

However, a left-shift-by-1 on 64-bit integers is equivalent to addition with itself: $x+x = 2x = x \ll 1$. The implementation exploits this identity to use the high-throughput vector integer addition unit instead of the shift unit:

```
#define ROT63(x) _mm256_xor_si256(_mm256_srli_epi64((x), 63), \
    _mm256_add_epi64((x), (x)))
```

This is both a latency and throughput optimization: on many μ architectures, vector integer addition has lower latency and more execution ports than vector variable shifts.

5.2.4 Explicit Round unrolling

All 20 rounds are explicitly unrolled in the source code, grouped as 5 blocks of 4 rounds each. This eliminates loop-counter arithmetic, branch-prediction pressure, and enables the compiler to schedule instructions across round boundaries. The round macros `ROUND_WITH_MSG` and `ROUND_WITHOUT_MSG` encapsulate the repeated pattern while preserving full unrolling.

5.2.5 Feed-forward with Register-saved Initial State

Instead of reloading the initial state from memory for the final feed-forward XOR, the initial values are saved in registers (`h0_init–h3_init`) at the beginning of the compression function. After 20

rounds, the output is computed as:

$$h_{\text{out}}^{(i)} = r_i \oplus h_{\text{init}}^{(i)}, \quad i \in \{0, 1, 2, 3\}$$

This avoids four additional 256-bit memory loads at the end of the critical path, and the saved registers remain live across all rounds—the compiler can spill them to memory at low pressure points if register pressure demands it.

5.2.6 Dual Buffering with Pointer Swapping

The top-level hash function maintains two 32-byte-aligned state buffers (`h_buf1`, `h_buf2`) and alternates between them using pointer swapping:

```
uint64_t (*h_curr)[4] = h_buf1;
uint64_t (*h_next)[4] = h_buf2;
// ... each block iteration ...
tmp = h_curr; h_curr = h_next; h_next = tmp;
```

This replaces an explicit `copy_state` (4×256 -bit SIMD loads and stores) per processed block with a simple pointer exchange. The `copy_state` function is only called once, during initialization from the IV.

5.2.7 Software Prefetching

A two-block-ahead software prefetch is issued at the beginning of each block iteration:

```
if (i + 2 < num_blocks)
    MM_PREFETCH(&padded[(i + 2) * BLOCK_BYTES]);
```

This uses the non-temporal hint to bring the message data into L1 cache before it is needed, hiding main-memory latency behind the computation of the current and next blocks.

5.2.8 Compiler Portability Abstraction

Platform-specific constructs are abstracted behind preprocessor macros:

- `ALIGN32` / `ALIGN32_ATTR`: 32-byte alignment directives for MSVC vs. GCC/Clang.
- `RESTRICT`: The `__restrict` qualifier, enabling the compiler to assume no pointer aliasing and perform more aggressive instruction reordering and vectorization.
- `MM_PREFETCH`: Prefetch intrinsic, mapped to `_mm_prefetch` on MSVC and `__builtin_prefetch` on GCC/Clang.

These abstractions allow the same source to compile optimally across compiler toolchains without duplication of the core algorithm.

5.3 Software Performance on 64-bit Platform

FEILIAN is easily implemented on 64-bit processors: it works on words of 64 bits, and only makes wordwise operations (XOR, rotation, addition) that are implemented in most of the processors. Because the core of FEILIAN is just the SubColumn (16 operations), implementations are simple and compact.

As requested by NGCC, we wrote a reference implementation and optimized implementations in C. Here we report speed benchmarks based on the optimized implementation. We compiled our program with gcc with options -O3 -march=native. We report speeds for various lengths of (aligned) messages, and give the median measurement over thousands of trials.

In Table 5.1, we report the hardware and software platforms of three 64-bit processors based on which we evaluate the performance of FEILIAN. And Table 5.2 reports the number of clock cycles required to generate one message digest with the full version of FEILIAN-1024, depending on the message length. Due to the design of FEILIAN hash family algorithm, similar results can also be obtained on FEILIAN-768 and FEILIAN-512. Table 5.3 presents the performance comparison data between FEILIAN-1024 and other hash functions obtained using the SUPERCOP framework from the eBACs project¹. It can be observed that our implementation achieves roughly equivalent software performance to SHA-512.

Table 5.1: Experimental Platforms (Hardware and Software)

Category	Component	Intel	AMD	ARM
Hardware	CPU Model	i7-10750H	Ryzen 9 8945HX	Apple M2 Pro
	Frequency	2.60GHz	2.50GHz	3.50GHz
	Instruction Set	AVX2	AVX2	NEON
	Core Count	1	1	1
	RAM	16GB	32GB	16GB
Software	Operating System	Windows 11	Windows 11	MacOS 26
	Compiler	GCC 16.1.0	GCC 15.2.0	Clang 17.0.0
	Build Tool	CMake 4.3.4	CMake 4.3.4	CMake 4.3.4

¹<https://bench.cr.yp.to/ebash.html>

Table 5.2: Performance of FEILIAN on different processors.

Data length [bytes]	10	100	1000	10000
Intel(R) Core(TM) i7-10750H				
FEILIAN-1024	56.98	5.77	3.65	3.57
AMD Ryzen 9 8945HX with Radeon Graphics				
FEILIAN-1024	44.15	4.71	3.09	3.05
Apple M2 Pro				
FEILIAN-1024	55.76	5.64	4.19	4.09

Table 5.3: Performance Comparison of BLAKE-64, SHA-512, SHA-3-512 and FEILIAN obtained by using the SUPERCOP framework from the eBACS (eBACS: ECRYPT Benchmarking of Cryptographic Systems) project (the first row of each algorithm records total clock cycles, and the second row denotes cycles per byte)

Data length [bytes]	64	128	576	1024	1536	4096	64576
BLAKE-64	425	775	1875	3331	4775	11237	171850
	6.640625	6.054688	3.255208	3.252930	3.108724	2.743408	2.661205
SHA-512	669	1106	2406	4175	5950	14712	222144
	10.453125	8.640625	4.177083	4.077148	3.873698	3.591797	3.44004
SHA-3-512	969	1375	4325	6850	9775	24456	376569
	15.140625	10.742188	7.508681	6.689453	6.363932	5.970703	5.831408
FEILIAN-1024	556	556	2425	3825	5675	15031	234819
	8.6875	4.34375	4.210069	3.735352	3.694661	3.669678	3.63632

5.4 Hardware Implementation Architecture

FEILIAN processes a 1024-bit message block and maintains a 1024-bit chaining state represented as sixteen 64-bit words $v_0, v_1, \dots, v_{15}$. No pipeline circuits have been designed, due to the enormous resource requirements of pipelined solutions. Nonetheless, three architectures of the compression

function have been investigated to evaluate the relation between speed and area. Every implemented circuit reports to the basic block diagram of Figure 5.1.

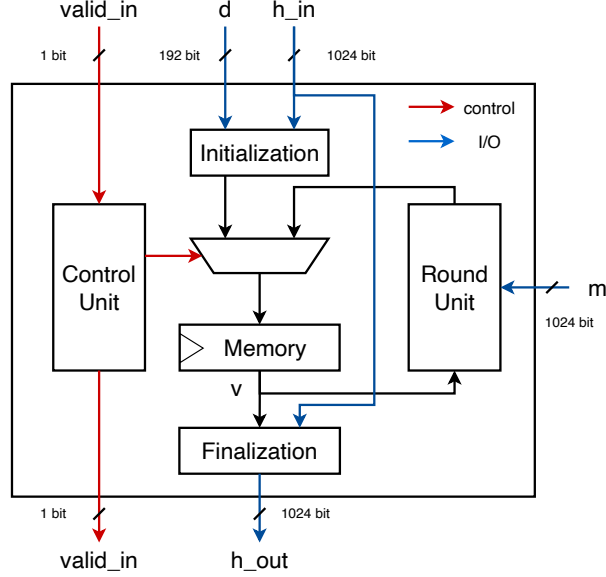


Figure 5.1: Block diagram of FEILIAN compression function $\text{FEILIAN-}f$.

Besides memory, the main modules of FEILIAN are

1. The initialization and finalization units, which are implemented in the compression module. The initialization unit contains sixteen 64-bit registers for the chaining value h_{in} , message block m , domain words, and internal state v . The finalization unit XORs the internal state after the final round with the latched input chaining value to produce the next chaining value.
2. The round function, which is implemented in the round module and instantiated by the compression module. A FEILIAN round contains two consecutive layers, and each layer consists of **SubColumns** followed by **ShiftRows**. One **SubColumns** layer applies four independent SubColumn operations to the four state columns; therefore, a full FEILIAN round contains eight logical SubColumn operations. Message injection is performed before each of the two **SubColumns** layers in every fourth round, and the same 1024-bit message block is reused at each injection point. Each SubColumn operation is applied to one column of four 64-bit state words and is composed of four modulo- 2^{64} adders and six XOR gates. Rotations are implemented as fixed rewiring of the internal word bits and therefore do not require additional logic.
3. The control unit, which is also implemented in the compression module. This unit orchestrates the compression-function computation using the I/O enable signals and a three-state control flow. It latches the chaining value, the message block, and the domain into local registers,

and initializes the internal state with the chaining value. The domain words are supplied by the upper module.

The following four architectures expose the natural area/speed scaling of the **FEILIAN** round function. All architectures use sixteen 64-bit state registers. The constants, round-control signals, message-word selection, and fixed row permutations are hard-wired control/datapath logic.

- [8SC]-**FEILIAN**. This architecture instantiates eight **SubColumn** units for one complete **FEILIAN** round. Four parallel **SubColumn** units compute the first **SubColumn** step, and four more **SubColumn** units compute the second **SubColumns** layer. One round is completed per cycle. This is the high-throughput architecture.
- [4SC]-**FEILIAN**. This architecture instantiates four parallel **SubColumn** units. Each cycle computes one **SubColumns** layer, followed by the corresponding **ShiftRows** wiring. A full round requires two cycles. This design is the main balanced architecture because it preserves column-level parallelism while reducing the round-unit area.
- [2SC]-**FEILIAN**. This architecture instantiates two parallel **SubColumn** units. Two cycles compute one **SubColumns** layer, followed by the corresponding **ShiftRows** wiring. A full round requires four cycles. This design targets compact hardware implementations.
- [1SC]-**FEILIAN**. This architecture instantiates a single **SubColumn** unit. The control unit selects one state column at a time and writes the updated words back to the state register file. Since a round contains eight **SubColumn** operations, a full round requires eight cycles.

5.5 Hardware Performance

We implement the **FEILIAN** compression function in SystemVerilog. The SystemVerilog source code is provided in Sec A.2. We report FPGA results as hardware implementation results. For FPGA, the area unit is the device-specific slice or LUT/FF count.

Experimental Setup

Table 5.4 summarizes the hardware and EDA tools used for FPGA implementation and evaluation.

Table 5.4: FPGA implementation platform.		
Category	Component	Specification
Hardware	FPGA Platform	Xilinx Virtex-5 xc5vlx110
		AMD Zynq-7000 xc7z100
EDA Tools	Synthesis Tool	Synplify Elite W-2024.09
	Synthesis Strategy	Base
	HDL Language	SystemVerilog

We evaluate the FPGA implementation of FEILIAN on two FPGA platforms, Xilinx Virtex-5 xc5vlx110 and AMD Zynq-7000 xc7z100. The design is synthesized using Synplify Elite W-2024.09 with the default synthesis strategy (i.e., base synthesis strategy).

The hardware evaluation focus on compression function. The 1024-bit message block is supplied in parallel to the compression function. Message buffering, padding logic, bit counters, and output registers are outside the measured block. Round constants and message selection logic are hard-wired in the datapath and are not stored in FPGA block RAM.

Area and Performance Results

Table 5.5 reports the synthesis results from Synplify.

Table 5.5: FPGA results for FEILIAN- f .					
Architecture	LUTs	FFs	Max Freq. [MHz]	Latency [us]	Throughput [Mbps]
Xilinx Virtex-5 xc5vlx110					
[8SC]-FEILIAN	5226	3419	60.0	0.35	2925.7
[4SC]-FEILIAN	3585	1057	84.3	0.48	2105.4
[2SC]-FEILIAN	5189	1561	78.0	1.04	986.1
[1SC]-FEILIAN	3964	2075	101.0	1.58	644.3
AMD Zynq-7000 xc7z100					
[8SC]-FEILIAN	5249	3429	85.0	0.24	4139.8
[4SC]-FEILIAN	3343	1142	142.8	0.28	3566.5
[2SC]-FEILIAN	5079	1558	114.2	0.71	1443.7
[1SC]-FEILIAN	3973	2083	139.3	1.16	886.0

The scalable structure of FEILIAN allows the implementation to be tuned for different hardware targets. The [8SC] and [4SC] architectures should be used to evaluate high-throughput use cases, such as communication-security datapaths. The [2SC] and [1SC] architectures reduce the number of instantiated SubColumn units, but their additional state storage, multiplexing, and control logic limit the expected area savings. Further area reduction depends on reducing the overhead of the scheduling logic, state storage, and state-update multiplexing.

Comparison with other hash functions. We compare [4SC]-FEILIAN with the BLAKE-512, SHA-512 and Keccak-512 in Table 5.6 on the same FPGA platform (Xilinx Virtex 5). The slice counts are reported for comparison, they are estimated from the LUT and FF counts as

$\text{slices} = \max(\lceil \text{LUTs}/4 \rceil, \lceil \text{FFs}/4 \rceil)$ according to the configuration of slices in the corresponding FPGA platform.

Table 5.6: Comparison with the baselines on the Xilinx Virtex 5.

Core	Area	Throughput	Efficiency
	[slices]	[Mbps]	[Mbps/slices]
BLAKE-512 [8]	406	563	1.39
SHA-512 [8]	818	2267	2.77
Keccak-512 [8]	1498	8056	5.38
FEILIAN-1024	897	2105	2.35

Chapter 6

Usage of FEILIAN

The indistinguishability established above implies that **FEILIAN** can securely replace a random oracle in cryptographic constructions. We now illustrate three applications: Message Authentication Codes (MAC), Key Derivation Functions (KDF), and Extendable Output Functions (XOF).

6.1 MAC

Length-extension attacks plague traditional Merkle–Damgård hashes, forcing constructions like HMAC to use nested hashing with two distinct keys. The built-in counter and final-block indicator **flag** render length-extension infeasible, enabling simpler and more efficient MAC designs.

A most straightforward construction is to prepend the key K to the message:

$$\text{MAC}_K(M) = \text{FEILIAN}(K \parallel M).$$

The indistinguishability guarantee ensures that **FEILIAN** behaves as a random oracle, so the prefix-key MAC is secure up to 512-bit security as a birthday bound of 1024-bit state of **FEILIAN** compression function **FEILIAN- f** . In particular, such a prefix-MAC is well suited for MAC based on **FEILIAN-512**.

Another natural construction is to use HMAC with **FEILIAN** as the underlying hash function, which is very effective due to the wide deployment of HMAC and the fast performance feature of **FEILIAN** in both software and hardware.

6.2 KDF

Standard Key-Derivation Functions (KDF), such as HKDF, follow an extract-then-expand paradigm. **FEILIAN** naturally accommodates both phases.

Given an input keying material IKM (which may have high entropy but non-uniform distribution), we extract a pseudo-random key:

$$\text{PRK} = \text{FEILIAN}(\text{IKM}).$$

The indistinguishability of **FEILIAN** ensures that the output is indistinguishable from a random bit value.

To produce multiple independent keys of desired length, we use the PRK as the salt for expansion:

$$K_i = \text{FEILIAN}(\text{PRK} \parallel \text{info} \parallel i), \quad i = 1, 2, \dots$$

where **info** is a context string with respect to application scenarios, and i is a counter encoded as part of the message. Since each K_i is a fresh random-oracle output on a distinct input (**info** $\parallel i$), the derived keys are pseudorandom and mutually independent. The total output can be arbitrarily long by iterating i .

6.3 XOF

An XOF allows arbitrary-length output by extending the squeezing phase. **FEILIAN** can be adapted to this setting with minimal changes.

Specifically, we set the parameter **ver** and **flag** for absorbing message and squeezing outputs separately, as below:

absorbing :	flag = 0x0000000000000000;
squeezing :	flag = 0xFFFFFFFFFFFFFFFF;

Table 6.1: The parameter **ver** of XOF

	absorbing	squeezing
FEILIAN-512:	0x00000000000000200	0xFF00000000000200
FEILIAN-768:	0x0000000000000300	0xFF00000000000300
FEILIAN-1024:	0x0000000000000400	0xFF00000000000400

We define a variant XOF as follows. First, process the input message M as in **FEILIAN** with **ver** and **flag** set as the absorbing value. We obtain the final chaining value h_{root} after processing all message blocks. To generate output blocks of length n , we continue the compression function calls with an empty message block, use the squeezing value of **ver** and **flag**, and re-initialize the counter to 0:

$$\text{Out}_0 = \text{FEILIAN}(h_{\text{root}}, \text{empty block}, t = 0),$$

$$\text{Out}_1 = \text{FEILIAN}(h_{\text{root}}, \text{empty block}, t = 1),$$

and so on. The final output is the concatenation as below:

$$\text{XOF}(M) = \text{Out}_0 \parallel \text{Out}_1 \parallel \dots$$

The empty message block is padded according to **FEILIAN** rules, and the counter ensures that each squeeze call is a distinct query to the compression function.

Acknowledgments

We thank the designers of existing hash functions and symmetric-key primitives more generally, in particular SHA-2, BLAKE, KECCACK, SM3, AES and Chacha, which inspired the design of FEILIAN hash family. In a high-level view, we did not reinvent the mode or the structural components. We investigated studied modes and components, deeply evaluated plausible combinations for various tradeoff consideration and searched concrete parameters for optimized integration.

We thank Dawu Gu and Yipeng Shi for valuable comments.

Bibliography

- [1] Nist statistical test suite for random and pseudorandom number generators for cryptographic applications. <https://csrc.nist.gov/Projects/Random-Bit-Generation/>.
- [2] Kazumaro Aoki and Yu Sasaki. Preimage attacks on one-block md4, 63-step MD5 and more. In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *Selected Areas in Cryptography, 15th International Workshop, SAC 2008, Sackville, New Brunswick, Canada, August 14-15, Revised Selected Papers*, volume 5381 of *Lecture Notes in Computer Science*, pages 103–119. Springer, 2008.
- [3] Jean-Philippe Aumasson. Too much crypto. *IACR Cryptol. ePrint Arch.*, 2019:1492, 2019.
- [4] Eli Biham and Orr Dunkelman. A framework for iterative hash functions - HAIFA. *IACR Cryptol. ePrint Arch.*, 2007:278, 2007.
- [5] Alex Biryukov and David A. Wagner. Slide attacks. In Lars R. Knudsen, editor, *Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24-26, 1999, Proceedings*, volume 1636 of *Lecture Notes in Computer Science*, pages 245–259. Springer, 1999.
- [6] John Black, Phillip Rogaway, and Thomas Shrimpton. Black-box analysis of the block-cipher-based hash-function constructions from PGV. In Moti Yung, editor, *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, volume 2442 of *Lecture Notes in Computer Science*, pages 320–335. Springer, 2002.
- [7] Ivan Damgård. A design principle for hash functions. In Gilles Brassard, editor, *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, Lecture Notes in Computer Science, pages 416–427. Springer, 1989.
- [8] Ekawat Homsirikamol, Marcin Rogawski, and Kris Gaj. Throughput vs. area trade-offs in high-speed architectures of five round 3 SHA-3 candidates implemented using xilinx and altera fpgas. In Bart Preneel and Tsuyoshi Takagi, editors, *Cryptographic Hardware and Embedded Systems - CHES 2011 - 13th International Workshop, Nara, Japan, September 28 - October 1, 2011. Proceedings*, volume 6917 of *Lecture Notes in Computer Science*, pages 491–506. Springer, 2011.
- [9] Sebastiaan Indesteege, Nathan Keller, Orr Dunkelman, Eli Biham, and Bart Preneel. A practical attack on keeloq. In Nigel P. Smart, editor, *Advances in Cryptology - EUROCRYPT*

- 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. *Proceedings*, volume 4965 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2008.
- [10] Antoine Joux. Multicollisions in iterated hash functions. application to cascaded constructions. In Matthew K. Franklin, editor, *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, volume 3152 of *Lecture Notes in Computer Science*, pages 306–316. Springer, 2004.
 - [11] John Kelsey and Tadayoshi Kohno. Herding hash functions and the nostradamus attack. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings*, volume 4004 of *Lecture Notes in Computer Science*, pages 183–200. Springer, 2006.
 - [12] John Kelsey and Bruce Schneier. Second preimages on n -bit hash functions for much less than 2^n work. In Ronald Cramer, editor, *Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings*, *Lecture Notes in Computer Science*, pages 474–490. Springer, 2005.
 - [13] Dmitry Khovratovich and Ivica Nikolic. Rotational cryptanalysis of ARX. In Seokhie Hong and Tetsu Iwata, editors, *Fast Software Encryption, 17th International Workshop, FSE 2010, Seoul, Korea, February 7-10, 2010, Revised Selected Papers*, volume 6147 of *Lecture Notes in Computer Science*, pages 333–346. Springer, 2010.
 - [14] Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, *Lecture Notes in Computer Science*, pages 218–238. Springer, 1989.
 - [15] Ivica Nikolic, Lei Wang, and Shuang Wu. Cryptanalysis of round-reduced md4 . In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 112–129. Springer, 2013.
 - [16] David A. Wagner. The boomerang attack. In Lars R. Knudsen, editor, *Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24-26, 1999, Proceedings*, volume 1636 of *Lecture Notes in Computer Science*, pages 156–170. Springer, 1999.

Appendix A

Source Code

A.1 C

We provide our C implementation of FEILIAN compression function FEILIAN-*f*.

Reference code

```
/* Single column nonlinear S-box transform */
static inline void sbbox_column(uint64_t *a, uint64_t *b, uint64_t *c,
    uint64_t *d)
{
    *a += *b;
    *c += *d;
    *d ^= *a;    *d = rotr64(*d, 8);
    *c += Sigma0_sbox(*d);
    *b ^= *c;    *b = rotr64(*b, 63);
    *a += Sigma1_sbox(*b);
}

/* Round with message block mixing */
static void round_with_msg(uint64_t state[MATRIX_ROWS][MATRIX_COLS],
    const uint64_t m[16],
    const uint64_t rc[MATRIX_COLS],
    const uint64_t flag_and_bits[MATRIX_COLS])
{
    // Inject first half message words
    for (int c = 0; c < MATRIX_COLS; c++) {
        state[0][c] ^= m[c];
        state[2][c] ^= m[4 + c];
    }
    sbbox_matrix(state);
    shift_rows(state);
    // Inject second half message words
```

```

    for (int c = 0; c < MATRIX_COLS; c++) {
        state[0][c] ^= m[8 + c];
        state[2][c] ^= m[12 + c];
    }
    sbbox_matrix(state);
    shift_rows(state);
    // XOR round constants & metadata bits
    for (int c = 0; c < MATRIX_COLS; c++) {
        state[1][c] ^= rc[c];
        state[3][c] ^= flag_and_bits[c];
    }
}

/* Core compression function */
void feilian_compress(uint64_t h_out[MATRIX_ROWS][MATRIX_COLS],
                     const uint64_t h_in[MATRIX_ROWS][MATRIX_COLS],
                     const uint64_t w_in[WORDS_PER_BLOCK],
                     int is_last_block,
                     const uint64_t hash_bits[2])
{
    uint64_t flag_and_bits[MATRIX_COLS];
    uint64_t version64 = FEILIAN_VERSION;
    // All-1 flag for last message block
    uint64_t flag64 = is_last_block ? UINT64_C(0xFFFFFFFFFFFFFFFF)
                                     : UINT64_C(0x0000000000000000);
    // Metadata vector: [version, last-block flag, len-mid, len-lo]
    flag_and_bits[0] = version64;
    flag_and_bits[1] = flag64;
    flag_and_bits[2] = hash_bits[0];
    flag_and_bits[3] = hash_bits[1];

    uint64_t h_initial[MATRIX_ROWS][MATRIX_COLS];
    copy_state(h_initial, h_in);
    copy_state(h_out, h_in);
    // Cyclic round constant groups
    const uint64_t *rc_groups[5] = { RC1, RC2, RC3, RC4, RC1 };

    /* Run total 20 rounds */
    for (int rnd = 0; rnd < TOTAL_ROUNDS; rnd++) {
        const uint64_t *rc = rc_groups[rnd / 4];
        // Every 4 rounds: mix message; pure transform otherwise
        if (rnd % 4 == 0) {
            round_with_msg(h_out, w_in, rc, flag_and_bits);
        } else {
            round_without_msg(h_out, rc, flag_and_bits);
        }
    }

    /* Feed-forward: XOR with input state */
    add_state(h_out, h_initial);
}

```

```
}
```

Optimized code

```
static void feilian_compress(hashState * h_in, hashState * h_out, uint8
    block, int is_last_block){
#define ROT63(x) _mm256_xor_si256(
    _mm256_srli_epi64((x), 63),
    _mm256_add_epi64((x), (x)))
#define SB0X_SIMD(r0, r1, r2, r3) do {
    __m256i _a_sbox = _mm256_add_epi64((r0), (r1));
    __m256i _c_sbox = _mm256_add_epi64((r2), (r3));
    __m256i _d_sbox = _mm256_xor_si256((r3), _a_sbox);
    _d_sbox = ROT8(_d_sbox);
    __m256i _s0_a = ROT5(_d_sbox);
    __m256i _s0_b = ROT48(_d_sbox);
    __m256i _sigma0 = _mm256_xor_si256(_d_sbox,
        _mm256_xor_si256(_s0_a, _s0_b));
    _c_sbox = _mm256_add_epi64(_c_sbox, _sigma0);
    __m256i _b_sbox = _mm256_xor_si256((r1), _c_sbox);
    _b_sbox = ROT63(_b_sbox);
    __m256i _s1_a = ROT11(_b_sbox);
    __m256i _s1_b = ROT40(_b_sbox);
    __m256i _sigma1 = _mm256_xor_si256(_b_sbox,
        _mm256_xor_si256(_s1_a, _s1_b));
    (r0) = _mm256_add_epi64(_a_sbox, _sigma1);
    (r1) = _b_sbox;
    (r2) = _c_sbox;
    (r3) = _d_sbox;
}while (0)

/* initialization */
uint64_t flag64 = is_last_block ? UINT64_C(0xFFFFFFFFFFFFFFFF)
    : UINT64_C(0x0000000000000000);

const __m256i fab = _mm256_set_epi64x(
    (long long)bits_lo, (long long)bits_mid,
    (long long)flag64, (long long)version64);

const __m256i rc1 = LOADU(RC1);
const __m256i rc2 = LOADU(RC2);
const __m256i rc3 = LOADU(RC3);
const __m256i rc4 = LOADU(RC4);

__m256i r0 = LOADU(&h_in[0][0]);
__m256i r1 = LOADU(&h_in[1][0]);
__m256i r2 = LOADU(&h_in[2][0]);
__m256i r3 = LOADU(&h_in[3][0]);
```

```

const __m256i h0_init = r0;
const __m256i h1_init = r1;
const __m256i h2_init = r2;
const __m256i h3_init = r3;

/* get message */
const __m256i m0 = LOADU(block + 0);
const __m256i m1 = LOADU(block + 32);
const __m256i m2 = LOADU(block + 64);
const __m256i m3 = LOADU(block + 96);

/* 20 rounds ( 5 phases ) compression */

/* phase 0 */
ROUND_WITH_MSG(r0, r1, r2, r3, m0, m1, m2, m3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);

/* phase 1 */
ROUND_WITH_MSG(r0, r1, r2, r3, m0, m1, m2, m3, rc2, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc2, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc2, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc2, fab);

/* phase 2 */
ROUND_WITH_MSG(r0, r1, r2, r3, m0, m1, m2, m3, rc3, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc3, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc3, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc3, fab);

/* phase 3 */
ROUND_WITH_MSG(r0, r1, r2, r3, m0, m1, m2, m3, rc4, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc4, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc4, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc4, fab);

/* phase 4 */
ROUND_WITH_MSG(r0, r1, r2, r3, m0, m1, m2, m3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);
ROUND_WITHOUT_MSG(r0, r1, r2, r3, rc1, fab);

/* FeedForward (XOR) */
STOREU(&h_out[0][0], _mm256_xor_si256(r0, h0_init));
STOREU(&h_out[1][0], _mm256_xor_si256(r1, h1_init));
STOREU(&h_out[2][0], _mm256_xor_si256(r2, h2_init));
STOREU(&h_out[3][0], _mm256_xor_si256(r3, h3_init));
}

```

A.2 SystemVerilog

We provide RTL implementation of the compression function of FEILIAN- f . One combinational round unit contains eight G datapaths and the compression function executes one round per cycle for 20 cycles.

The package fixes the compression-function parameters and keeps constants, rotations, and packing helpers in a single place. The packed state and message layout is little-endian by 64-bit word: word i occupies $[i*64 +: 64]$.

```
package feilian_pkg;
    localparam int CHAIN_BITS   = 1024;
    localparam int STATE_BITS   = 1024;
    localparam int BLOCK_BITS   = 1024;
    localparam int DOMAIN_BITS  = 256;
    localparam int ROUNDS       = 20;
    localparam int WORD_BITS    = 64;
    localparam int WORDS        = 16;
    localparam int GROUPS       = 5;
    localparam int INJECT_INTERVAL = ROUNDS / GROUPS;

    typedef logic [WORD_BITS-1:0] word_t;
    typedef word_t state_t [0:WORDS-1];

    localparam word_t ROUND_CONSTANTS [0:3] = '{
        64'h6a09e667f3bcc908, 64'hbb67ae8584caa73b,
        64'h3c6ef372fe94f82b, 64'ha54ff53a5f1d36f1
    };

    localparam word_t VERSION = 64'h400;
    localparam int DOMAIN_VERSION_WORD = 0;
    localparam int DOMAIN_FLAG_WORD   = 1;

    function automatic state_t unpack_words(
        input logic [WORDS*WORD_BITS-1:0] packed_words
    );
        for (int i = 0; i < WORDS; i++) begin
            unpack_words[i] = packed_words[i*WORD_BITS +: WORD_BITS];
        end
    endfunction

    function automatic logic [STATE_BITS-1:0] pack_state(input state_t
        words);
        pack_state = '0;
        for (int i = 0; i < WORDS; i++) begin
            pack_state[i*WORD_BITS +: WORD_BITS] = words[i];
        end
    endfunction
endpackage
```

The Subcolumn module is combinational.

```
module subcolumn
    import feilian_pkg::*;
    (
        input  logic [3:0][WORD_BITS-1:0] state_i,
        output logic [3:0][WORD_BITS-1:0] state_o
    );

    word_t a;
    word_t b;
    word_t c;
    word_t d;

    localparam int ROTR_D = 8;
    localparam int ROTR_B = 63;

    localparam int SIGMA0_R1 = 5;
    localparam int SIGMA0_R2 = 48;
    localparam int SIGMA1_R1 = 11;
    localparam int SIGMA1_R2 = 40;

    function automatic word_t rotr_d(input word_t x);
        rotr_d = {x[ROTR_D-1:0], x[WORD_BITS-1:ROTR_D]};
    endfunction

    function automatic word_t rotr_b(input word_t x);
        rotr_b = {x[ROTR_B-1:0], x[WORD_BITS-1:ROTR_B]};
    endfunction

    function automatic word_t sigma0(input word_t x);
        sigma0 = x
            ^ {x[SIGMA0_R1-1:0], x[WORD_BITS-1:SIGMA0_R1]}
            ^ {x[SIGMA0_R2-1:0], x[WORD_BITS-1:SIGMA0_R2]};
    endfunction

    function automatic word_t sigma1(input word_t x);
        sigma1 = x
            ^ {x[SIGMA1_R1-1:0], x[WORD_BITS-1:SIGMA1_R1]}
            ^ {x[SIGMA1_R2-1:0], x[WORD_BITS-1:SIGMA1_R2]};
    endfunction

    always_comb begin
        a = state_i[0] + state_i[1];
        c = state_i[2] + state_i[3];
        d = rotr_d(state_i[3] ^ a);
        c = c + sigma0(d);
        b = rotr_b(state_i[1] ^ c);
        a = a + sigma1(b);
    end
end
```

```

    assign state_o[0] = a;
    assign state_o[1] = b;
    assign state_o[2] = c;
    assign state_o[3] = d;
endmodule

```

The round unit is combinational. It unpacks the 1024-bit state and message into sixteen 64-bit words, optionally xors message words `m[0..7]` before the first `SubColumns` layer, applies four *G* units, shifts rows, then optionally xors `m[8..15]` before the second `SubColumns` layer. The four round constants and four domain words are injected at the end of each round.

```

module round
    import feilian_pkg::*;
    (
        input  logic [STATE_BITS-1:0] v_in,
        input  logic [BLOCK_BITS-1:0] m,
        input  logic [DOMAIN_BITS-1:0] d,
        input  logic [ROUND_GROUP_BITS-1:0] round_group,
        input  logic inject_en,
        output logic [STATE_BITS-1:0] v_out
    );

    function automatic state_t xor_message(
        input state_t words,
        input state_t msg,
        input int unsigned base
    );
        xor_message = words;
        for (int col = 0; col < 4; col++) begin
            xor_message[col] = words[col] ^ msg[base + col];
            xor_message[col + 8] = words[col + 8] ^ msg[base + 4 + col];
        end
    endfunction

    function automatic state_t shift_rows(input state_t words);
        shift_rows[0] = words[0];
        shift_rows[1] = words[1];
        shift_rows[2] = words[2];
        shift_rows[3] = words[3];
        shift_rows[4] = words[5];
        shift_rows[5] = words[6];
        shift_rows[6] = words[7];
        shift_rows[7] = words[4];
        shift_rows[8] = words[10];
        shift_rows[9] = words[11];
        shift_rows[10] = words[8];
        shift_rows[11] = words[9];
        shift_rows[12] = words[15];
        shift_rows[13] = words[12];
        shift_rows[14] = words[13];
        shift_rows[15] = words[14];
    endfunction

```



```

endfunction

function automatic state_t add_round_constants(
    input state_t words,
    input logic [ROUND_GROUP_BITS-1:0] round_group,
    input logic [255:0] d
);
    int unsigned rc_index;

    add_round_constants = words;
    for (int i = 0; i < ROUND_CONSTANT_WORDS; i++) begin
        rc_index = (int')(round_group) + i) % ROUND_CONSTANT_WORDS;
        add_round_constants[4 + i] = words[4 + i] ^ ROUND_CONSTANTS[
            rc_index];
    end
    add_round_constants[12] = words[12] ^ d[DOMAIN_VERSION_WORD*
        WORD_BITS +: WORD_BITS];
    add_round_constants[13] = words[13] ^ d[DOMAIN_FLAG_WORD*WORD_BITS
        +: WORD_BITS];
    add_round_constants[14] = words[14] ^ d[2*WORD_BITS +: WORD_BITS];
    add_round_constants[15] = words[15] ^ d[3*WORD_BITS +: WORD_BITS];
endfunction

state_t v_words;
state_t msg_words;
state_t phase0_words;
state_t phase0_sub_words;
state_t phase1_words;
state_t phase1_sub_words;
state_t result_words;
logic [3:0][WORD_BITS-1:0] phase0_g_in [0:3];
logic [3:0][WORD_BITS-1:0] phase0_g_out [0:3];
logic [3:0][WORD_BITS-1:0] phase1_g_in [0:3];
logic [3:0][WORD_BITS-1:0] phase1_g_out [0:3];
genvar g_col;

generate
    for (g_col = 0; g_col < 4; g_col++) begin : gen_phase0_g
        subcolumn u_subcolumn (
            .state_i(phase0_g_in[g_col]),
            .state_o(phase0_g_out[g_col])
        );
    end

    for (g_col = 0; g_col < 4; g_col++) begin : gen_phase1_g
        subcolumn u_subcolumn (
            .state_i(phase1_g_in[g_col]),
            .state_o(phase1_g_out[g_col])
        );
    end
end

```

```

endgenerate

always_comb begin
    v_words = unpack_words(v_in);
    msg_words = unpack_words(m);

    for (int col = 0; col < 4; col++) begin
        phase0_g_in[col] = '0;
        phase1_g_in[col] = '0;
    end
    for (int i = 0; i < WORDS; i++) begin
        phase0_sub_words[i] = '0;
        phase1_sub_words[i] = '0;
    end

    phase0_words = v_words;
    if (inject_en) begin
        phase0_words = xor_message(phase0_words, msg_words, 0);
    end
    for (int col = 0; col < 4; col++) begin
        phase0_g_in[col][0] = phase0_words[col];
        phase0_g_in[col][1] = phase0_words[col + 4];
        phase0_g_in[col][2] = phase0_words[col + 8];
        phase0_g_in[col][3] = phase0_words[col + 12];

        phase0_sub_words[col] = phase0_g_out[col][0];
        phase0_sub_words[col + 4] = phase0_g_out[col][1];
        phase0_sub_words[col + 8] = phase0_g_out[col][2];
        phase0_sub_words[col + 12] = phase0_g_out[col][3];
    end
    phase0_words = shift_rows(phase0_sub_words);

    phase1_words = phase0_words;
    if (inject_en) begin
        phase1_words = xor_message(phase1_words, msg_words, 8);
    end
    for (int col = 0; col < 4; col++) begin
        phase1_g_in[col][0] = phase1_words[col];
        phase1_g_in[col][1] = phase1_words[col + 4];
        phase1_g_in[col][2] = phase1_words[col + 8];
        phase1_g_in[col][3] = phase1_words[col + 12];

        phase1_sub_words[col] = phase1_g_out[col][0];
        phase1_sub_words[col + 4] = phase1_g_out[col][1];
        phase1_sub_words[col + 8] = phase1_g_out[col][2];
        phase1_sub_words[col + 12] = phase1_g_out[col][3];
    end
    phase1_words = shift_rows(phase1_sub_words);

    result_words = add_round_constants(phase1_words, round_group, d);

```

```

        v_out = pack_state(result_words);
    end
endmodule

```

The compression function module receives the chaining value, message block, and 256-bit domain. The control signals are `valid_in` and `valid_out`. Internally it uses three states: IDLE, RUN, and OUTPUT.

```

module compression
    import feilian_pkg::*;
    #(
        parameter int ROUNDS = feilian_pkg::ROUNDS,
        parameter int CHAIN_BITS = feilian_pkg::CHAIN_BITS,
        parameter int BLOCK_BITS = feilian_pkg::BLOCK_BITS,
        parameter int DOMAIN_BITS = feilian_pkg::DOMAIN_BITS
    ) (
        input logic clk,
        input logic rst,
        input logic start,
        input logic [CHAIN_BITS-1:0] h_in,
        input logic [BLOCK_BITS-1:0] m,
        input logic [DOMAIN_BITS-1:0] d,
        output logic done,
        output logic [CHAIN_BITS-1:0] h_out
    );
    typedef enum logic [1:0] {
        IDLE,
        RUN,
        OUTPUT
    } state_t;
    localparam int ROUND_IDX_W = $clog2(ROUNDS);
    localparam int ROUND_OFFSET_W = $clog2(INJECT_INTERVAL);
    localparam logic [ROUND_IDX_W-1:0] LAST_ROUND = ROUNDS - 1;
    localparam logic [ROUND_OFFSET_W-1:0] LAST_ROUND_OFFSET = INJECT_INTERVAL
        - 1;
    localparam logic [ROUND_GROUP_BITS-1:0] LAST_ROUND_GROUP =
        ROUND_CONSTANT_WORDS - 1;

    state_t state;
    state_t state_nxt;
    logic [CHAIN_BITS-1:0] h_reg;
    logic [CHAIN_BITS-1:0] h_nxt;
    logic [BLOCK_BITS-1:0] m_reg;
    logic [BLOCK_BITS-1:0] m_nxt;
    logic [DOMAIN_BITS-1:0] d_reg;
    logic [DOMAIN_BITS-1:0] d_nxt;
    logic [CHAIN_BITS-1:0] v_reg;
    logic [CHAIN_BITS-1:0] v_nxt;
    logic [ROUND_IDX_W-1:0] round_reg;
    logic [ROUND_IDX_W-1:0] round_nxt;

```

```

logic    [ROUND_OFFSET_W-1:0] round_offset_reg;
logic    [ROUND_OFFSET_W-1:0] round_offset_nxt;
logic    [ROUND_GROUP_BITS-1:0] round_group_reg;
logic    [ROUND_GROUP_BITS-1:0] round_group_nxt;
logic    [CHAIN_BITS-1:0] round_v;
logic    inject_msg;
logic    [ROUND_GROUP_BITS-1:0] round_group;

round u_round (
    .v_in      (v_reg),
    .m         (m_reg),
    .d         (d_reg),
    .round_group(round_group),
    .inject_en(inject_msg),
    .v_out     (round_v)
);

assign done      = (state == OUTPUT);
assign h_out     = v_reg;
assign inject_msg = (round_offset_reg == '0);
assign round_group = round_group_reg;

always_comb begin
    state_nxt = state;
    h_nxt     = h_reg;
    m_nxt     = m_reg;
    d_nxt     = d_reg;
    v_nxt     = v_reg;
    round_nxt = round_reg;
    round_offset_nxt = round_offset_reg;
    round_group_nxt = round_group_reg;

    case (state)
        IDLE: begin
            if (start) begin
                h_nxt     = h_in;
                m_nxt     = m;
                d_nxt     = d;
                v_nxt     = h_in;
                round_nxt = '0;
                round_offset_nxt = '0;
                round_group_nxt = '0;
                state_nxt = RUN;
            end
        end
        RUN: begin
            if (round_reg == LAST_ROUND) begin
                v_nxt     = round_v ^ h_reg;
                state_nxt = OUTPUT;
            end else begin

```

```

        v_nxt      = round_v;
        round_nxt = round_reg + 1'b1;
        if (round_offset_reg == LAST_ROUND_OFFSET) begin
            round_offset_nxt = '0;
            round_group_nxt  = (round_group_reg == LAST_ROUND_GROUP) ? '0 :
                                round_group_reg + 1'b1;
        end else begin
            round_offset_nxt = round_offset_reg + 1'b1;
        end
    end
end
OUTPUT: begin
    state_nxt = IDLE;
end
default: begin
    state_nxt = IDLE;
end
endcase
end
always_ff @(posedge clk) begin
    state      <= rst ? IDLE : state_nxt;
    h_reg      <= h_nxt;
    m_reg      <= m_nxt;
    d_reg      <= d_nxt;
    v_reg      <= v_nxt;
    round_reg   <= rst ? '0 : round_nxt;
    round_offset_reg <= rst ? '0 : round_offset_nxt;
    round_group_reg <= rst ? '0 : round_group_nxt;
end
endmodule

```

Appendix B

Test Vectors

We provide intermediate values for hashing a one-block and a two-block message, for each of the digest sizes (1024-bit, 768-bit, 512-bit). For the one-block case, we hash the 24-bit message “abc”(0x616263). For the two-block case, we hash the 1032-bit message “aaa...aaa”(0x61...61). Values are given left to right, top to bottom. For example

```

0000000000000000  0000000000000000  0000000000000000  0000000000000000
0000000000000000  0000000000000000  0000000000000000  0000000000000000
0000000000000000  0000000000000000  0000000000000000  0000000000000000
0000000000000000  0000000000000000  0000000000000000  0000000000000000

```

represents

```

m0   m1   m2   m3
m4   m5   m6   m7
m8   m9   m10  m11
m12  m13  m14  m15

```

B.1 FEILIAN-1024

One-block message: “abc”(24-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

Message block after padding:

6162630000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

8334de7038cdcb85	1ed8141136e28928	01f91f2586257b36	e144101a72c1b40d
20a41f6053903a10	aba2702e4678c28b	b534895121a43005	af9b86363b90286f
a4bb3345dc6194ed	2b4763887f4d9534	772e0ec74cef7f04	c04630135814353b
be8a831ae6878daa	f39b181cdf4ed4d	a8400088d9b51f91	07b30666c664c155

State v after 2 rounds:

d587089462e3a8f7	4980ed99aed55e7d	da074737728d10a7	c62953768351c69f
5e6bb6896289c22e	f2c5dfd08e16ed9c	fd52f668fa0f6054	ed1fd74c0a580008
136c04ebb030d2e8	be146d9cf4581dfe	7d01e0ca34eaa0ad	74f2cac2dc69625f
225b77f7db554df3	9822320820498c10	2eb4565194ce68d7	ee6ffc4c79c8f9ea

State v after 5 rounds:

1779a7d2caac92e2	ba9bccb36a5240db	90ad2b1f9822fcb2	32799993f29a1f76
38a5339509125330	b0d99d7dc8286a2e	258df88346641808	7a3f658398fa04a0
c76c566d95a861fb	781b9e4707b6a9f1	d6b782c412f83af8	417e1183fc94272a
0b49898cb9863f35	2224937e8c79b7d1	561e627b2bf0ee16	f69a865b75bfda04

State v after 10 rounds:

570f66f4dcc3df5f	2bef50bde438fd83	b66f4d21465cebec	a91544eda2ac87af
006e3aa0b76c25fc	713ac7a5b5681811	03d08b63ed7e5d48	48cdff7299042e48
81932c3875c3245b	a82ee7c3dc7503b3	1fcdbff9477c8f3e	6f5798e35d398bb7
9391a645337a3005	a25083a3c061e10a	a565e397450ac87b	c0cdbbc9a869bd552

State v after 20 rounds:

01ef1f6734470fd6	54350424e973d09b	4022f91f795af629	23c31b291c5bea5a
e150b6244682d579	4354f396f9aab25a	456680cc69647723	3695e0f26a364bde
81b7febe65d188c7	071c0ef0c6cade85	6717dded76fa7f8e	74635d0f5311a51e
c9a27ffb2ebf7243	ef179a6f67184067	bac2196af3ffb86f	e2afc6c6b20267c4

Hash digest output:

0507e4b1ef75d025	dfa303ea0a8e2c47	f9c7c5503dc12be4	d38615f0b1e1ed2b
0ec6527ec29778a4	36be43cd599500fd	fe2718a07ba9ca85	c94271df47351109
dc73a8ec672ba113	296b155e56052dd6	39a0e0a636afea48	88db3739e2f282cc
da0d93dfbeefde73	902c89d42803b6cb	7944717688ebc3b2	ad2955c31ee6c681

Two-block message: “aaa...aaa”(1032-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

First compression Message block after padding:

6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

f01ee65fe2d995e1	b87199dd1a3aec43	730814128df1c391	4f479a3de6ee33b7
530a6d360ae12f66	ffbcecf646a8b7387	4128ba2fd6f4ff16	e45e159cbdbe30b2
05481641294fc182	2ef13c65a49d2eda	01e308fd335f4bfa	f268931426754e37
1e1f080ddab1a612	46564839925aa873	e8a30359ab8933a3	d8265c3432469996

State v after 2 rounds:

5b7ecf694c4ed7a5	ecb84988deac61e2	a268c3892fa397ab	6f7677cd9ff86ab6
8fe9dc870ac04000	b13ff5a5818bfa54	b34012831d152260	f0e9abf5e776afe9
f17c472c9e6bdbce	5ba566ce432c281a	7b1e52e34511f9f4	e4d398ca26c69a76
d433e5621b120e89	8d5d81ce383c134f	5c9def7941a91110	7ba3521432594576

State v after 5 rounds:

5d244c2f7d69df60	4575772b5ef377b5	7221fcfd041137e3	d5fa5f6fca21e43f
6164b571939f5fde	3b31cf1ccda2c3f0	5cefe0755ce935bb	dc27ad1c32a32057
0d2c7445672599d8	52c749065043d57c	1dd498193ef19315	cb5aef76f3747a5c
f4d6de53975b9873	35235db9ad6d4aee	29e87da08c90570d	bca0cd7063f7127e

State v after 10 rounds:

9c6e086be1262b41	81ce566ecbf83980	f23a7c4c7c251306	ff39cc547f6b9346
b78e18a2edbf1022	bf00d6b30dd663fe	1d6168b74d0e1f18	f6fe6eb0cb95357d
83a4d6f4fe2e0f58	1267e4181b4d29df	823e7f80a84814d8	50c56ca0db853ff0
7d1216a347721943	7ed529a7e62176dc	5776a9a4c8698737	cc547aa52640671e

State v after 20 rounds:

25980f537295fe01	759f692ca4092ff6	2e9079932f61b2bc	2d02159089e696d9
8e7330cad89a90ed	56243236f7582bf0	1e627f55bf488891	4303095b4d5cd276
dff773c465a249ff	a26b5974ad96991b	3b86a9e9f5496d00	c8a4a66c909725f2
a93c4397a956cacd	1f2a7e5bad045a35	91448666a3b58989	eb9382b5fc4d14a3

Second compression Message block after padding:

6100000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

01a765dbf736f6d2	6686e302a7795cb2	8a9941b106fe836c	252cef0865a8fa50
cb5b112ce04a839a	e87054f9c3b1279c	dece56e27634d84c	7c87dceef81bdb61
4de1a61decdbb2e4	735a52d235492cb7	147bdb322553b2b7	70450981fab15b64
1340d3d2587ab554	3b8be71c1e9536c2	99457484263b759f	88faa26d8d1a5aca

State v after 1 round:

3422323f5fa49c69	2f98c87861ca8460	6de6fae43b6a3194	e30824faacecf2ed
f80cda1f169229ea	f93133365460a814	9967f9e75736a3be	e6ca57c3c4ea50b3
d796627738cb0255	4d9f6f4f8d9226f0	d67a9aeb13dfff7e	b7543b550840ff02
37870439e7296584	2d451abc85bc33de	912138349c03a5bf	13d50d01a411a653

State v after 2 rounds:

d797d92ba06091f7	bbc0da9adf2d5b20	4247344f3dbd4cfb	0111148b78afe830
846f4aa11141f7a3	5363357b59f253b8	9bf9282b5f6e7570	98f9679f466c9a1b
752b8395a50a5b9d	c242aa864bdc4544	418b56bd666e7ebc	1c5f481a09c01a5c
2f46582e97184c61	b0dd8a7bc1b85b64	236adc95656acf4c	ea11a71c570e866a

State v after 5 rounds:

a73eb7bcd12aeb92	68e0f5ede6cbc255	3f69f5dd752eedf	f11f7e9d1cb57fa5
b3ddf9f07da39069	87e6c893ef364f95	b35ab4df197eeba3	29a415a30f1b471b
eb58b853c07a3969	c8ecb816c81b118b	f53298097fd2be92	cd79e1bc7d23baac
7178ac85790635f2	1782a843b581d1a5	faa8f5f0a9ddedd1	57598f42f6c47ac5

State v after 10 rounds:

a376c8f66a58e2de	cd452fb7adf9782a	5fa4b16ec5de1fdf	2bb4ed90763efec3
1942ef63a12b0a7d	bdbb504879605a1d	b9d2bd9b78a40a97	82d6753de422a371
d6bdb6b32475a878	948ae0e2c68c22e9	e86447b7ccaafc67e	62ab19a243d2dd56
31bc7aa1d6985062	30126d800d8ee360	7f3a5f10796d1af5	78dd23339a8d5615

State v after 20 rounds:

ec958fbbc50bce31	d8509fb05f6fbf3d	8b45ed8bc297dace	dabc3e14e33cf666
5e05e8c7e4652b0f	e37fef900a46c985	61284a679ea99ba2	310f89e1eea1fadf
538ccd4294b8f04b	be05e897347c3fc0	0f6dadd1c7b6d15c	f5a7009e0b3fa139
735b0d6404c13bff	c92018cff9b74f93	c31a86f4b91d0fcc	6d946556959e2edb

Hash digest output:

e3383d3260ea32ed	8fe316f8b27cd6be	a25969c43aacdc01	360c94861cd190ff
95a82f04ebf95e95	19eef7c969bb0f0b	ee439de8851ce6bf	be21ba160f55884d
af4263785f6b6d1e	7713350145ba5fcd	eb63e5e2e376161b	5dfa8ef11f09e285
ab8ebb5cb6de1b60	517922e7d3ffabf2	537a269f70f25f5a	117484183bc76ee5

B.2 FEILIAN-768

One-block message: “abc”(24-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

Message block after padding:

6162630000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

8334de7038cdcb85	1ed8141136e28928	01f91f2586257b36	e144101a72c1b40d
20a41f6053903a10	aba2702e4678c28b	b534895121a43005	af9b86363b90286f
a4bb3345dc6194ed	2b4763887f4d9534	772e0ec74cef7f04	c04630135814353b
be8a831ae6878aaa	f39b181cdcf4ed4d	a8400088d9b51f91	07b30666c664c54d

State v after 2 rounds:

7f646b7c248841a2	674b63e7d4e87d4d	573884a1d15d84c3	9040ec47c4110a8a
731bb68962919054	5ca3d7d06e9dc61d	ee491bc88101a4a2	ac5f4eaa995eb3de
95df00ebc07d9f10	47999b4cc9d5458e	5da1ac397d69f946	624acac2dc654b62
743cc1de831d75fc	588d44021640a229	29b4565194ce68d7	b63e7c4c79c8ea3a

State v after 5 rounds:

de21c355a1b950be	553d9add74aacc2	3ca4167f02eb5e3d	556678a4f0feb511
2f3812773723cfa1	b798a47345878735	4daceaddcc0bdc46	05eeaf36b572e75d
fcaca424dd81aef0	32ec58b967abdd41	b7532568609be8e4	8bb191cacc71079f
fa02830cd06d85b6	b2f7d96592f350b8	a75e56eb86f874ea	ba752cad77f20d6

State v after 10 rounds:

95488e223247f4f4	9d96ca8afb6c12dd	f2a692a2d1be6580	e1229f94a3db639d
f0778c0f4a120ffc	29b23aac47f83756	96ec3336ff0538f1	9d84e530b42d8b46
bc3e8dee12d54c8d	b1b8d54eed6b9b8	29c4feae0af0ac37	ed85390a093c5af9
c925a5d7d66086ba	37c5237c654c4360	6370870c6192e9e6	3a7a82fe15e4a39c

State v after 20 rounds:

ff4e91e0df509d63	36ec27ce643c82ff	d07c066f6d084190	a1c46500ce3faefc
450d87b55f8ef52a	ff352036c7870627	5666def8b9c55888	271f8bb73849ae92
187bd64f6cda780c	4e49d17e1b6e7f77	b1188e9afa4196f8	26561c44123c76ba
bcfa9a224cb720ba	544db3a6dfc420b4	09b08396c075ce7b	89349a50106f86b2

Hash digest output:

b095f35a68fb71db	bbf14c67e0adf525	407097444d3e7574	75c27122989feaa9
5de65e6753a62500	4b0a6ef3f9466141	5508b9704ff7ca96	85a70e8d025e9b18
1783a3e596036d8a	dbcab183d8da789f	4f495b2a41fce59e	2c081a78a9b3b79e

Two-block message: “aaa...aaa”(1032-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Round constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

First compression Message block after padding:

6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

f01ee65fe2d995e1	b87199dd1a3aec43	730814128df1c391	4f479a3de6ee33b7
530a6d360ae12f66	ffbecf646a8b7387	4128ba2fd6f4ff16	e45e159cbdbe30b2
05481641294fc182	2ef13c65a49d2eda	01e308fd335f4bfa	f268931426754e37
1e1f080ddab1a112	46564839925aa873	e8a30359ab8933a3	d8265c3432469996

State v after 2 rounds:

4f22afb9bc7ded04a	42a89b8ee0c207e1	719ac97f2635a262	436119afcaa088d0
9259dc870ac05a02	213ff5a5818de05e	77171283093dec22	7ce940e6fcffd905
b97c472c9e68d6cb	c98ee6ce493e4524	3d1e276ac8d54282	ea0b98ca26c69777
f703e5621b120f9f	d3ac6fc82635b971	5b9def7941a91110	7ba3521432594576

State v after 5 rounds:

2f350c8c80932dce	fc8c86925e12dcf1	ac703cc97da1ef85	47a48601c5090aad
27a41ae47ed7e0b7	d46bcd03bfd21e09	a147d50154a3fc39	464019c15db36abb
04d5dc12cc7699ec	10f16180fad74210	09bd3ee87b73289a	e7c5670da7996fa2
f3508f17095a0370	fb903842f1ceee1	ef04a6459a78ab65	4820c865c1cdeb5a

State v after 10 rounds:

a23f518428e69cc5	8816d372f1b46506	ba5e591f4be36aaa	90803de36d5ce791
5ae491d1e0789e28	01131699d65f3790	fb79613be0dc1222	802957197c31782d
749d8e43850c472b	a58c8fde1d643e8c	f401c3983ee6ba4d	d0910066232514d0
605bcb51fc8eed54	162e00c5935e56e5	f1889ab7fda298e5	5a197f3376f39de6

State v after 20 rounds:

2366222144d88011	38d7c27410f2ccd9	1329d67e403a39c2	6db63c5281146b58
3343cb5bd7611109	d2a48fc8e8b8f428	c42aa9edf7f0935d	8c200e7c622b1895
efe774d37c76146b	ffd3357186a4cb12	bbac650a34caf38a	9ddc7a2d6ad62e45
e28d288be04a5183	5aa7ddc112ffc0b7	c2ab80a744259a58	3ae20af2cd3913bc

Second compression Message block after padding:

6100000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

075948a9c17b88c2	2bce485a1382bf9d	b720ee5c69a50812	6598c6ca6d5a07d1
766beabdefb1027e	6cf0e907dc51f844	0486805a3e8cc380	b3a4dbc9d76c1182
7df1a10af50fef70	2ee23ed71e7b7ebe	945117d1e4d02c3d	253dd5c000f050d3
58f1b8ce11662e1a	7e064486a16eac40	caaa7245c1ab664e	598b2a2abc6e5dd5

State v after 1 round:

cf338bfaeb153338	ea65bd2fe9e4f36f	672887b6172d14dd	b0c273fcedfcc11c
17591bc9b6e2e6ac	04a66b88d4cb6372	91f15bc5ee402fa9	613a21538376ae9d
9f2f4e4b278df2fc	40ae1df1de754d82	5ed2e6a632250ee8	0ab17e093603a9fa
c11996a97b9a0bf4	d656bd13d8ec8785	76f9ee871a3d5954	328a64f3b1b85fb5

State v after 2 rounds:

cd3c340ac9d48ad0	792062fc8c56030a	ae8080ce9ca43f9e	da8daad72b4c3666
b0a5b41f33c13025	bbc89f6b02ad0ae7	e56ad45bdb1028dd	5b836f0cf0a34eb3
adfc9b957de8b1e26	a9f17669b09ae509	ef0eb650dad244ee	ad72ec379ec76358
06391cc7540fce82	76ff23bf23420073	58e16f7dc1844cca	573cee67bc9f822d

State v after 5 rounds:

cbb6981f6b7adb9d	d57c38916f68a236	36b0ee73ba0c85d2	6477253246555914
c4685a606a8edfd9	e4fbf75fa18570e6	1d1aeeb4724a0e1f	a4c1f9831033b5b4
6db6af1202d5224a	58852cdfc1449ad3	5b508f3ef1bf1e77	fa53f7d06ce870fc
4f124b4b694b4b20	499da53e4f8e4158	602715a9160a1859	d32f8bf030d6d8ba

State v after 10 rounds:

f8f694f77fa111ad	55fefaf5802e474d1	672e6a1f65eb0df7	3661627f1f5ff549
9f0b50541d1f7603	eed1b03f9534e506	dff1d54b2bdc5d6f	8cbe215fc93510c9
f7ff25a3850018ce	c7d509258dbae269	e069b81805bbfce0	1ddc46c0dc62eb20
55f714608506fd03	61613cda9f5dc146	2f420c407ce44e9f	4216a428fff065d8

State v after 20 rounds:

00c180279655dec2	b3e207355516f0bc	93e44bec78ed9022	6cbfb364c4f8e459
c37cdcd590c1fc59	2c9be4f36ae283c0	87ddd3e550022d1c	cd079b9339cfbc9d
77c7f01cbc30cf4e	81b4cb3e259e3a1b	8dcf80ac7f48304e	7b715a0554dbcfad
c13590592803b835	b31825790eb787fc	a37b9b8509157b56	6b89ce11ef473a40

Hash digest output:

00562e578ec89807	214f94466f4f2c98	30984811b0a5c424	88e3a2a9ae752709
27fe707f683617b5	847bb3b6f40d6b40	9cee8e6ebf535b83	1fada3ee5a40a37e
3e203f491651360a	a544e53be9f556af	731c989b7d979e19	7e9f2b54c58f4c5e

B.3 FEILIAN-512

One-block message: “abc”(24-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Round constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

Message block after padding:

6162630000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

8334de7038cdcb85	1ed8141136e28928	01f91f2586257b36	e144101a72c1b40d
20a41f6053903a10	aba2702e4678c28b	b534895121a43005	af9b86363b90286f
a4bb3345dc6194ed	2b4763887f4d9534	772e0ec74cef7f04	c04630135814353b
be8a831ae6878baa	f39b181cdf4ed4d	a8400088d9b51f91	07b30666c664c54d

State v after 2 rounds:

a0c151a81b29cdf4	aa1b21dddae67f46	06f67eabcf6b268e	4fadecacb7ff4e28
710bb68962919e52	4ca3d7d06e9bb81f	aa0a1bcb07895c11	9f5f3a36fa127f22
9ddf00ebc07ea011	f5b81b4d0aef07d1	442196774ccf9f38	6342cac2dc654c61
760cc1de831d3616	195d865c244a101e	28b4565194ce68d7	b63e7c4c79c8ea3a

State v after 5 rounds:

b65d0573f2209ba6	9c52a4d8ed9eb1	41a81a52e118ae63	b3471ab2b77dc622
03f63f2b13ccf183	3ec26ecdeedc43e7	92eec6993a4f1d48	e7fbe12863b99895
e745ea3263b89992	72e41b2e23e20f7e	7cfbadda31207162	76c2e80683046882
6a012055487181e4	b2059c3c8c4b4819	8b8ea619fd9e6f45	126122463f325bff

State v after 10 rounds:

ecf3ccf978b80922	02ac9177d533b7d5	032a73f0758696ed	359ab3da6051e97a
76790ad44feb9268	b6ed71f6e6f17e99	410f95e3ffb8d523	0ad1fd3160b83b32
e9c904e61ab766b2	a5162afdbbdcc4d4	7787b4db76b89d3b	8c8a8b30c1b8ce74
60b81fce5cb6bd42	61ab79dc511b8bd8	6839c81cc6c1a2a5	0b2b46774580f134

State v after 20 rounds:

26cb47b9ea6b9765	7b2e214f3f542eb9	8cd7dfd0e4244355	84d4926a973e5b6e
aa92c16124859399	bf706442eda3af03	2aaf6df838ca6578	00b9e5f871ba689b
68761e997bc73e88	1dc987c3962787ab	1ec2998b9b0ab10e	30d250c269c3e3d8
875e9dab5b93126e	cbdf8ecf9bc74b02	1b6c109e0fc04edc	ed1460b8fab2ce8c

Hash digest output:

b69fc86f312df402	fd5d243c61ab3768	8572bbcdf2e7de28	e737707bf268fa8c
ee80551c87e0baef	6fa34ad98d022401	a535b6f14f4403ea	8c61fdc44d303d3f

Two-block message: “aaa...aaa”(1032-bit)

IV:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

Round constants:

6a09e667f3bcc908	bb67ae8584caa73b	3c6ef372fe94f82b	a54ff53a5f1d36f1
------------------	------------------	------------------	------------------

First compression Message block after padding:

6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161
6161616161616161	6161616161616161	6161616161616161	6161616161616161

Initial state of v:

243f6a8885a308d3	452821e638d01377	9216d5d98979fb1b	ba7c9045f12c7f99
13198a2e03707344	be5466cf34e90c6c	d1310ba698dfb5ac	24a19947b3916cf7
a4093822299f31d0	c0ac29b7c97c50dd	2ffd72dbd01adfb7	0801f2e2858efc16
082efa98ec4e6c89	3f84d5b5b5470917	b8e1afed6a267e96	636920d871574e69

State v after 1 round:

f01ee65fe2d995e1	b87199dd1a3aec43	730814128df1c391	4f479a3de6ee33b7
530a6d360ae12f66	ffbecf646a8b7387	4128ba2fd6f4ff16	e45e159cbdbe30b2
05481641294fc182	2ef13c65a49d2eda	01e308fd335f4bfa	f268931426754e37
1e1f080ddab1a012	46564839925aa873	e8a30359ab8933a3	d8265c3432469996

State v after 2 rounds:

09e1b9f5b8db61fd	80d85d84e2bffdfc	81dcc77d2b879c24	ae3959a3e5cce4d4
9049dc870ac0441c	d13ff5a5818fe650	5ad6128315c5d822	6979f4ce4bea3700
c17c472c9e69d5cc	cf6e66ce47404126	b7d67d7e935fb580	eb0398ca26c69878
f913e5621b120c9d	10fc2dca4c33b752	5a9def7941a91110	7ba3521432594576

State v after 5 rounds:

819468fa37ed37f2	7cc2b600423c6823	a30139671d97d34c	3d98ec37e819cc99
78c6568b866a7d64	ec21ffd4abb85378	9c001c91ab2fa140	856efc8969f394bc
f4b20575375a5637	52cf79be1a5d0a12	400e32435df649b4	5de1a857303ed12a
97d994b75b56e3f7	5f9ae9b565a0c4bb	f3a1068d6eed8f52	7e3b881a8cdb1e1b

State v after 10 rounds:

ccc854f8e730c7ec	e9cac6c586ae0f32	5c6923e75747fa2d	8da7f43d3d0d54fa
1ec27ff4054c5727	87f56825c1558267	c5539ead21105104	48073bbb3a9e0eab
d2ed080d226832d1	3b8ef33dfc66bddd	a9b8354273c287db	c08f7937a2971696
1fae93f2bbad794d	6fe986461bf11bd6	927f4fd5fe6f8ac9	f910396da5d13e04

State v after 20 rounds:

9e88570a0fe9d140	9e4991ccb3ee30e7b	e3394f218a2469e1	b7518f67057051fc
28d8f9ca6a4bf9c0	6943192cc0e15162	e7a49d1e3c2071d2	87837fa8b8b8e147
d170b90cd631743c	4eacf18997dd20a6	0e3575c2a66d4b1a	93fecc89f187c2a2
f00a7a2739b1e419	df1dfddbb6f60827	8fe7e41f30866210	3224a82590b64dde

Second compression Message block after padding:

6100000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000

Initial state of v:

bab73d828a4ad993	8d501be2bd937d3f	47307703a3bb5831	bf7f75ffe93e3d75
6df0d82c529beab7	d7177fe3f4085d0e	2708b4a9f55c210f	b807aa1d0dffe850
43666cd55f488f27	9f9dfa2f0f02950a	21c80719767794ad	2b1f63649ba1bc34
4a76ea62c89d9b80	fbbc649c056764d0	87e616fdb5089e06	514d88fde1e103b7

State v after 1 round:

1b92ba0b8ed9901e	322a6b7f6dbd961a	6f0d49fb7c9a3a93	74a0cb3fb47cfa1d
390c7d5bbc3ba40c	1492d64cfdcdff7f	7bdcae148043d613	1aebd1e247d78cce
28edfcc54a09cb4b	c15e284378787b6f	ea31511506e6663e	fc01b326cef2dd4
7acb3590069a33ac	0b769eaa73cc8aa8	f74b0b30628d4b86	029d6c134202ae50

State v after 2 rounds:

3997f897738d7264	27b0dc1245c2a115	27aa95af46e825f6	9735d72ed367270c
93c9c1cf4b6cb421	44b0d903ffa61493	a60bee4c41c043c2	4a599fb020f948a6
97e7be0a85f0b639	9c847ca1a6f5dd7d	41758cd34890b4d6	718cd67de1b5f5f2
f29d5170f4ddd1a1	65ce716253370974	13b547584216ce75	b22bbb780b459c5b

State v after 5 rounds:

899e8207f6607ea8	8799d5e5a5545db2	397cc1a08f80c9e8	5f1dc6d4c870afaa
5ffa715e10fa4ebb	7fcaa102cace2b96	c41ce5245283161d	2860fbc84c7827e0
f9050f5d1e0992f2	bbbe5a4b9e10870c	656e93cfdea6e795	dd1794d3b8495044
cc33144bfea4ac37	7201ae6d2d5003a7	825df5b4648495ba	d5aea41083691228

State v after 10 rounds:

a3a66d718393b190	57a641b6f559ca28	14ae98ca821165c1	964924180c0c2aec
70e5c9163b9bb7b8	75f591d991ff9075	e032c9a487a0ebe5	5232eab7184dc473
4ba41a1226552766	1317a3c2f1febb79	106b9cceb5f54179f	99a74034e10ea0d9
b4873adf9a5f8b08	b1110e17b9cbd657	2299b7ce044bdad0	eb2bde5268202f16

State v after 20 rounds:

b72bcb92da14a15c	438457a2083168f6	414a18d028582920	428b125f3a580ced
685fe170c68bdb74	47308caeb842ceab	eb94c299dc518afa	f4b8ca3ea11d82a9
212bf03ff7964073	4ff8d3a44387a998	7c0e7f06c70c1f9d	b407a8379acee379
1bff5606cae55701	5ae9993390e82acd	cfbd7f3e95e202d1	20fe47339ed1512a

Hash digest output:

cf785e5010f69c0d	c915a2b5404cd4ce	1171e38bd36f7a06	983166d3a067f4fd
c33110945c39af05	a5934a4c4df32790	f5ab0d2930769ccc	f96ae2ac2360bf4c